

Cấu trúc dữ liệu và thuật toán mạng

Robert Endre Tarjan

SIAM, 1983

Nguồn gốc: Data Structures and Network Algorithms

English Materials / Data Structures and Network Algorithms - Tarjan

Tóm tắt

Đây là ghi chú dịch tiếng Việt cho chuyên khảo ngắn của Robert E. Tarjan về mối liên hệ giữa cấu trúc dữ liệu, phân tích thuật toán và tối ưu mạng. Bản dịch trong kho này chuyển ngữ phần thông tin đầu sách, mục lục và lời nói đầu, đồng thời ghi lại nội dung chính của từng chương để phục vụ tra cứu trong bối cảnh luyện thi thuật toán.

Thông tin sách

Sách thuộc bộ *CBMS-NSF Regional Conference Series in Applied Mathematics*, do SIAM xuất bản. Bản gốc:

Tác giả	Robert Endre Tarjan
Cơ quan	Bell Laboratories, Murray Hill, New Jersey
Nhà xuất bản	Society for Industrial and Applied Mathematics
Năm	1983
ISBN	0-89871-187-8

1 Mục lục đã dịch

Chương 1. Nền tảng Giới thiệu; độ phức tạp tính toán; cấu trúc dữ liệu nguyên thủy; ký hiệu thuật toán; cây và đồ thị.

Chương 2. Các tập rời nhau Tập rời nhau và cây nén; cận trên khẩu hao cho nén đường đi; ghi chú.

Chương 3. Heap Heap và cây có thứ tự heap; d -heap; heap trái; ghi chú.

Chương 4. Cây tìm kiếm Tập có thứ tự và cây tìm kiếm nhị phân; cây nhị phân cân bằng; cây nhị phân tự điều chỉnh.

Chương 5. Nối và cắt cây Bài toán nối và cắt cây; biểu diễn cây thành các tập đường; biểu diễn đường bằng cây nhị phân; ghi chú.

Chương 6. Cây khung nhỏ nhất Phương pháp tham lam; ba thuật toán cổ điển; thuật toán vòng tròn; ghi chú.

Chương 7. Đường đi ngắn nhất Cây đường đi ngắn nhất, gán nhãn và quét; thứ tự quét hiệu quả; mọi cặp đỉnh.

Chương 8. Luồng mạng Luồng, lát cắt và đường tăng; tăng theo luồng chặn; tìm luồng chặn; luồng chi phí nhỏ nhất.

Chương 9. Ghép cặp Ghép cặp hai phía và luồng mạng; đường luân phiên; blossom; thuật toán cho ghép cặp không hai phía.

2 Lời nói đầu

Trong mười lăm năm trước khi cuốn sách này được viết, lĩnh vực thuật toán tổ hợp đã phát triển bùng nổ. Dù phần lớn công trình mới có tính lý thuyết, nhiều thuật toán vừa được khám phá lại rất thực dụng. Các thuật toán ấy không chỉ dựa trên kết quả mới trong tổ hợp và đặc biệt là lý thuyết đồ thị, mà còn dựa trên sự phát triển của cấu trúc dữ liệu mới và kỹ thuật phân tích thuật toán mới.

Mục tiêu của tác giả là làm rõ sự tương tác giữa các hướng này bằng cách giải thích những thuật toán hiệu quả nhất đã biết cho một số bài toán tổ hợp chọn lọc. Sách bao quát bốn bài toán kinh điển trong tối ưu mạng, kèm theo việc xây dựng các cấu trúc dữ liệu cần dùng và phân tích thời gian chạy. Tác giả dự định đưa phần này vào một bộ sách hai tập toàn diện hơn về cấu trúc dữ liệu và thuật toán đồ thị.

Ba mục tiêu được nêu rõ là chiều sâu, độ chính xác và sự đơn giản. Tác giả cố gắng trình bày các kỹ thuật tiên tiến nhất lúc bấy giờ theo cách dễ hiểu và có thể dùng trong thực tế. Điều ông muốn truyền đạt là vẻ đẹp và chiều sâu của thuật toán đồ thị, kiến thức về các thuật toán tốt nhất cho những bài toán được xét, và cách hiện thực chúng.

Nội dung sách dựa trên các bài giảng tại hội nghị CBMS Regional Conference ở Worcester Polytechnic Institute tháng 6 năm 1981. Sách cũng bao gồm công trình chưa công bố khi đó của Tarjan cùng Daniel Sleator tại Bell Laboratories. Tác giả cảm ơn ban tổ chức hội nghị, các học viên tham gia, National Science Foundation, những người chuẩn bị bản thảo, Michael Garey vì các nhận xét sắc sảo, và đặc biệt là Dan Sleator.

3 Chương 1: Nền tảng

Chương đầu đặt khung cho toàn bộ sách. Tarjan xét các thuật toán hiệu quả cho bốn bài toán tối ưu mạng kinh điển. Những thuật toán này kết hợp hai mảng: cấu trúc dữ liệu và phân tích thuật toán ở một bên, tối ưu mạng ở bên kia. Tối ưu mạng lại liên quan tới vận trù học, khoa học máy tính và lý thuyết đồ thị.

Một điểm nhấn quan trọng là: nhiều trình bày về thuật toán mạng bỏ qua chi tiết cấu trúc dữ liệu, khiến người muốn cài đặt thuật toán phải tự giải một bài lập trình không nhỏ. Tarjan chọn cách trình bày ở mức các thao tác đơn giản trên những đối tượng toán học nguyên thủy như danh sách, cây và đồ thị, thay vì viết chương trình cụ thể bằng FORTRAN.

Chương này cũng nhắc tới vai trò của độ phức tạp tính toán như một dạng dao cạo Occam. Thuật toán hiệu quả nhất thường là thuật toán chỉ tính đúng lượng thông tin cần thiết cho tình huống bài toán. Vì vậy, phát triển một thuật toán đặc biệt hiệu quả thường đem lại hiểu biết sâu hơn về chính bài toán; kết quả không chỉ nhanh mà còn đơn giản và đẹp.

Giới thiệu và mô hình độ phức tạp

Sách khảo sát các thuật toán máy tính hiệu quả cho bốn bài toán tối ưu mạng kinh điển. Các thuật toán ấy ghép hai nguồn ý tưởng: cấu trúc dữ liệu và phân tích thuật toán ở một phía, tối ưu mạng ở phía kia. Tối ưu mạng lại lấy công cụ từ vận trù học, khoa học máy tính và lý thuyết đồ thị. Với các bài toán được chọn, mục tiêu không phải liệt kê mọi cách giải, mà là hiểu các thuật toán tốt nhất đã biết và vì sao chúng đạt được cận thời gian đó.

Tarjan giả sử người đọc đã có nền tảng nhập môn về cấu trúc dữ liệu, phân tích thuật toán, thuật toán đồ thị và tối ưu mạng. Điểm được nhấn mạnh là các thuật toán tốt nhất thường không xuất hiện nếu ta tách rời hai mảng này. Nhiều phần trình bày thuật toán mạng chỉ nói rất sơ về cấu trúc dữ liệu, khiến người cài đặt còn phải tự giải một bài toán lập trình lớn. Ngược lại, khi xét kỹ độ phức tạp tính toán, ta thường bị buộc phải nhận ra chính xác thông tin nào thật sự cần được duy trì; đó là lý do các thuật toán nhanh thường đồng thời sáng sủa và gọn.

Để nói về hiệu quả, trước hết cần chọn mô hình tính toán. Máy Turing là mô hình lịch sử và rất hữu ích cho lý thuyết độ phức tạp bậc cao: nó có bộ điều khiển hữu hạn, một băng nhớ hai chiều gồm các ô, và một đầu đọc/ghi mỗi bước có thể đọc, ghi, dịch trái hoặc phải, rồi đổi trạng thái. Nhưng mô hình này quá thô để phân tích chính xác các thuật toán thực dụng.

Một mô hình gần máy tính thông thường hơn là máy truy cập ngẫu nhiên. Nó có một chương trình hữu hạn, một tập thanh ghi, và bộ nhớ là mảng các ô có địa chỉ rõ ràng. Trong một bước, máy có thể thực hiện một phép toán số học hoặc logic trên thanh ghi, nạp nội dung của ô nhớ có địa chỉ nằm trong thanh ghi, hoặc ghi nội dung thanh ghi vào một ô nhớ như vậy.

Mô hình máy con trở yếu hơn một chút nhưng phù hợp với các thuật toán danh sách và cây. Bộ nhớ của nó là một tập nút có thể mở rộng, mỗi nút gồm một số trường đặt tên. Một trường chứa được một số hoặc một con trỏ tới nút khác. Muốn đọc hoặc ghi một trường của nút, máy phải đang giữ con trỏ tới nút đó trong thanh ghi. Mô hình này không cho phép số học địa chỉ, nên những kỹ thuật như băm không được biểu diễn trực tiếp; đổi lại, nó phản ánh tốt hơn các thuật toán xử lý cấu trúc liên kết và thuận tiện hơn khi chứng minh cận dưới.

Ba mô hình trên đều tuần tự và tất định: mỗi thời điểm chỉ thực hiện một bước, và trạng thái hiện tại quyết định duy nhất hành vi tiếp theo. Sách không đi vào tính toán song song hay bất định, dù các mô hình đó quan trọng trong lý thuyết và trong kiến trúc máy tính.

Có một cảnh báo quan trọng. Nếu máy truy cập ngẫu nhiên hoặc máy con trỏ được phép xử lý số nguyên có kích thước tùy ý trong thời gian hằng, ta có thể giấu tính toán song song bên trong một số nguyên lớn bằng cách mã hóa nhiều giá trị vào cùng một từ. Có hai cách tránh điều này. Một là dùng thước đo chi phí logarit, tính thời gian của phép toán theo số bit của toán hạng. Hai là giới hạn số nguyên được phép dùng chỉ có $O(\log n)$ bit, với n đo kích thước đầu vào, và hạn chế các phép toán trên số thực. Tarjan chọn cách thứ hai: các thuật toán trong sách có thể cài trên máy truy cập ngẫu nhiên hoặc máy con trỏ với số nguyên kích thước đa thức nhỏ theo n , không dựa vào mã hóa mẹo.

Sau mô hình máy, cần chọn thước đo độ phức tạp. Độ dài chương trình là thước đo tĩnh, hữu ích nếu chương trình chỉ chạy rất ít lần hoặc trong một số câu hỏi lý thuyết. Với mục tiêu ở đây, thước đo động như thời gian chạy và bộ nhớ theo kích thước đầu vào phù hợp hơn. Sách chủ yếu đo thời gian chạy; hầu hết thuật toán được xét có bộ nhớ tuyến tính theo kích thước đầu vào.

Khi phân tích thời gian, các hằng số thường được bỏ qua. Điều này vừa đơn giản hóa chứng minh, vừa giảm phụ thuộc vào chi tiết mô hình máy. Ký hiệu tiệm cận được dùng theo nghĩa chuẩn: nếu f và g là các hàm của các biến không âm, f là $O(g)$ nếu tồn tại các hằng số dương c_1, c_2 sao cho

$$f(n, m, \dots) \leq c_1 g(n, m, \dots) + c_2$$

với mọi giá trị của các biến. Ta viết f là $\Omega(g)$ nếu g là $O(f)$, và f là $\Theta(g)$ nếu đồng thời $f = O(g)$ và $f = \Omega(g)$.

Thời gian chạy thường được đo theo đầu vào xấu nhất. Cách này cho một bảo đảm chắc chắn, nhưng có thể bị quan nếu trường hợp xấu hiếm gặp. Phân tích trung bình trên phân phối đầu vào là một lựa chọn khác, nhưng khó hơn và phụ thuộc mạnh vào việc phân phối có phản ánh thực tế hay không. Một hướng bền hơn là cho thuật toán tự dùng lựa chọn ngẫu nhiên rồi lấy trung bình trên các lựa chọn đó, dù với các bài toán trong sách đây không phải con đường chính.

Loại trung bình quan trọng nhất trong sách là phân tích khấu hao. Khi một cấu trúc dữ liệu bị thao

tác lặp đi lặp lại, một thao tác riêng lẻ có thể đắt, nhưng tổng chi phí của cả dãy thao tác lại nhỏ hơn nhiều so với “số thao tác nhân với chi phí xấu nhất của một thao tác”. Cây nén trong union-find, cây tự điều chỉnh và các cấu trúc cây động đều dùng kiểu lập luận này.

Một thuật toán được gọi là hiệu quả nếu thời gian xấu nhất của nó bị chặn bởi một đa thức theo kích thước đầu vào. Một bài toán được gọi là xử lý được (*tractable*) nếu nó có thuật toán hiệu quả; tập các bài toán như vậy được ký hiệu là P . Lý do khái niệm này quan trọng là các thuật toán thời gian đa thức thường chậm đi từ từ khi kích thước tăng, còn thuật toán phi đa thức thường có một ngưỡng mà sau đó trở nên không dùng được, dù tăng tốc máy hay cho thêm thời gian một hằng số lần. Hơn nữa, thuật toán hiệu quả thường phản ánh một cấu trúc thật của bài toán, trong khi thuật toán không hiệu quả thường chỉ là vết cạn bị đánh bại bởi bùng nổ tổ hợp.

Cấu trúc dữ liệu nguyên thủy

Ngoài số nguyên, số thực và bit, sách coi một số đối tượng phức tạp hơn là nguyên thủy: khoảng, danh sách, tập và ánh xạ. Một khoảng $[j..k]$ là dãy số nguyên $j, j+1, \dots, k$. Ký hiệu cũng được mở rộng cho cấp số cộng: $[j, k..l]$ biểu diễn dãy bắt đầu tại j , bước nhảy $k-j$, và dừng trước khi vượt quá l . Ký hiệu $\in$ và $\notin$ dùng chung cho khoảng, danh sách và tập.

Một danh sách $q = [x_1, x_2, \dots, x_n]$ là một dãy phần tử, có thể có phần tử lặp. Phần tử đầu x_1 là *head*, phần tử cuối x_n là *tail*, và kích thước được viết là $|q| = n$. Ba phép toán nền tảng trên danh sách là:

- truy cập: $q(i)$ trả về phần tử thứ i , hoặc null nếu i nằm ngoài miền hợp lệ;
- lấy danh sách con: $q[i..j] = [x_i, x_{i+1}, \dots, x_j]$, với chỉ số thiếu được hiểu là biên tương ứng của danh sách;
- nối: nếu $r = [y_1, \dots, y_m]$, thì $q \& r = [x_1, \dots, x_n, y_1, \dots, y_m]$.

Các thao tác trên hai đầu danh sách tạo ra những cấu trúc quen thuộc. Truy cập đầu, *push* và *pop* tạo thành ngăn xếp, hoạt động theo kiểu vào sau ra trước. Truy cập đầu, *inject* ở cuối và *pop* ở đầu tạo thành hàng đợi, hoạt động theo kiểu vào trước ra trước. Nếu cả sáu thao tác ở hai đầu đều có, ta có hàng đợi hai đầu (*deque*); nếu thiếu thao tác lấy ở cuối, đó là deque bị hạn chế đầu ra.

Một tập $s = \{x_1, x_2, \dots, x_n\}$ là một bộ sưu tập phần tử phân biệt, không có thứ tự ngầm định. Các phép toán quan trọng là hợp, giao và hiệu. Nếu q là danh sách và s là tập, hiệu $q - s$ được hiểu là danh sách thu được từ q sau khi xóa mọi bản sao của các phần tử thuộc s .

Một ánh xạ $f = \{[x_1, y_1], \dots, [x_n, y_n]\}$ là một tập cặp có thứ nhất đôi một khác nhau. Miền xác định là các x_i , miền giá trị là các y_i , và $f(x_i) = y_i$. Nếu x không thuộc miền xác định thì $f(x) = \text{null}$. Phép gán $f(x) := y$ thay giá trị tại x , còn $f(x) := \text{null}$ xóa cặp tương ứng. Một danh sách cũng có thể nhìn như ánh xạ với miền $[1..|q|]$.

Ánh xạ có thể được biểu diễn bằng mảng giá trị nếu miền là một khoảng hoặc có thể đổi về một khoảng; nó cũng có thể là một trường trong nút nếu miền là tập nút. Hai cách nhìn này tương ứng với bộ nhớ của máy truy cập ngẫu nhiên và máy con trỏ. Sách dùng ký hiệu hàm $f(x)$ cho cả giá trị ánh xạ, giá trị trong mảng, trường của nút, hoặc kết quả hàm, vì về mặt thuật toán chúng đều là cách biểu diễn một hàm.

Một tập có thể biểu diễn bằng hàm đặc trưng trên một vũ trụ U : $\chi_s(x)$ đúng khi $x \in s$ và sai khi $x \in U - s$. Cách này cho phép kiểm tra thuộc tập và cập nhật thêm/xóa một phần tử trong $O(1)$ thời gian. Nhiều cấu trúc thực tế dùng hàm đặc trưng kèm với biểu diễn danh sách hoặc tập khác; nếu thường xuyên cần kích thước, ta duy trì riêng một biến đếm và cập nhật nó trong $O(1)$.

Danh sách có thể biểu diễn bằng mảng hoặc bằng cấu trúc liên kết. Với ngăn xếp, mảng kèm chỉ số ô cuối đang dùng là đủ để truy cập đầu, *push* và *pop* trong $O(1)$. Với deque, có thể giữ hai chỉ số đầu-cuối

và cho danh sách vòng qua cuối mảng bằng phép modulo. Cách này tốt khi biết cận trên khá sát cho kích thước lớn nhất và không cần nhiều thao tác cắt danh sách con hoặc nối, vì các thao tác đó có thể phải sao chép nhiều phần tử.

Các biểu diễn liên kết được phân loại theo ba trục: nội sinh hay ngoại sinh, liên kết đơn hay kép, tuyến tính hay vòng. Cấu trúc nội sinh đặt các con trỏ tạo khung ngay trong nút dữ liệu; cấu trúc ngoại sinh giữ khung bên ngoài các nút. Danh sách đơn có con trỏ tới phần tử kế tiếp; danh sách kép có thêm con trỏ tới phần tử trước. Danh sách tuyến tính kết thúc bằng null, còn danh sách vòng nối phần tử cuối về phần tử đầu. Danh sách đơn tuyến tính đủ cho ngăn xếp; danh sách đơn vòng hỗ trợ tốt deque bị hạn chế đầu ra và nối hủy đầu vào trong $O(1)$; danh sách kép vòng đủ cho deque đầy đủ.

Biểu diễn nội sinh tiết kiệm bộ nhớ hơn, nhưng thường yêu cầu một phần tử chỉ nằm trong một hoặc một số cấu trúc cố định tại một thời điểm. Một biến thể hữu ích là dùng nút giả ở đầu danh sách để khỏi xử lý riêng trường hợp rỗng. Nếu cần đảo danh sách nhanh, có thể dùng danh sách kép vòng với hai con trỏ trong mỗi nút không được đặt tên cố định là trước/sau, trừ nút đuôi. Vì mọi truy cập đi qua đuôi, ta xác định lại hướng khi duyệt và vẫn thực hiện các thao tác deque, nối và đảo trong $O(1)$ mỗi thao tác.

Ký hiệu thuật toán

Thuật toán trong sách được viết bằng mô tả từng bước hoặc bằng một ngôn ngữ giả gần Algol, kết hợp lệnh canh gác của Dijkstra và tinh thần tập hợp của SETL. Ký hiệu $:=$ là gán, dấu chấm phẩy tách câu lệnh. Sách cho phép gán tuần tự, gán song song và hoán đổi bằng mũi tên kép; gán song song được hiểu là mọi vế phải được tính từ trạng thái cũ trước khi biến ở vế trái đổi giá trị.

Ba cấu trúc điều khiển chính là `if...fi`, `do...od` và `for...rof`. Trong `if`, các điều kiện được xét và danh sách lệnh ứng với điều kiện đúng đầu tiên được thực hiện; nếu không điều kiện nào đúng thì không làm gì. `do` tương tự, nhưng sau khi chạy một nhánh, các điều kiện được xét lại cho tới khi mọi điều kiện đều sai. `for` chạy thân vòng lặp một lần cho mỗi giá trị của bộ lặp; nếu bộ lặp chạy trên danh sách thì thứ tự là thứ tự danh sách, nếu chạy trên tập thì thứ tự không dự đoán trước.

Ngôn ngữ cũng có thủ tục, hàm và vị từ. Tham số mặc định truyền theo giá trị; các biến thể khác là truyền kết quả và truyền vừa giá trị vừa kết quả. Khi tham số là tập, danh sách hoặc cấu trúc tương tự, ta hiểu rằng thứ được truyền là một con trỏ tới biểu diễn thích hợp của cấu trúc. Kiểu dữ liệu gồm số nguyên, số thực, bit, ánh xạ, danh sách, tập và các kiểu do người dùng định nghĩa; sách cố ý không nhấn mạnh cơ chế kiểm tra kiểu.

Một số hàm dựng sẵn được giả định. `create type` tạo một nút mới thuộc kiểu đã cho. `min s` và `max s` lấy phần tử nhỏ nhất/lớn nhất của một tập hoặc danh sách số; dạng “by key” chọn nút có khóa nhỏ nhất/lớn nhất theo một trường hoặc hàm. `sort s` trả về danh sách các phần tử của `s` theo thứ tự tăng; dạng “sort by key” sắp các nút theo khóa.

Ví dụ minh họa là merge sort trên danh sách. Thủ tục `merge(s, t)` trộn hai danh sách đã sắp bằng cách quét hai danh sách theo thứ tự không giảm, tốn $O(|s| + |t|)$. Để sắp danh sách `s`, ta biến mỗi phần tử thành danh sách một phần tử, đưa các danh sách con vào hàng đợi, rồi lặp lại bước lấy hai danh sách đầu hàng đợi, trộn chúng, và đưa kết quả về cuối hàng đợi. Mỗi lượt qua hàng đợi tốn $O(|s|)$ và gần như giảm một nửa số danh sách con, nên tổng thời gian là $O(|s| \log |s|)$, tối ưu đến hằng số đối với sắp xếp bằng so sánh. Nếu trước tiên tách `s` thành các đoạn đã tăng sẵn thay vì các phần tử đơn, ta được biến thể *natural list merge sort*.

Cây và đồ thị

Đối tượng chính của sách là cây và đồ thị. Một đồ thị $G = [V, E]$ gồm tập đỉnh V và tập cạnh E . Nếu đồ thị vô hướng, mỗi cạnh là một cặp không thứ tự của hai đỉnh phân biệt; nếu đồ thị có hướng, mỗi cạnh là

một cặp có thứ tự. Để tránh lặp định nghĩa, sách dùng (v, w) cho cả cạnh vô hướng $\{v, w\}$ và cạnh có hướng $[v, w]$, ngữ cảnh quyết định cách hiểu. Vòng lặp và đa cạnh không được xét trong định nghĩa cơ bản, nhưng các thuật toán đều mở rộng được cho chúng.

Trong đồ thị vô hướng, nếu $\{v, w\}$ là cạnh thì v và w kề nhau. Trong đồ thị có hướng, cạnh $[v, w]$ rời v và đi vào w . Hai đỉnh đầu mút của một cạnh được gọi là các đầu của cạnh; cạnh và hai đầu của nó là liên thuộc. Một cạnh liên thuộc với tập đỉnh S nếu chính xác một đầu của nó nằm trong S . Đồ thị là hai phía nếu tồn tại $S \subseteq V$ sao cho mọi cạnh đều có một đầu trong S và một đầu trong $V - S$. Bậc của đỉnh trong đồ thị vô hướng là số đỉnh kề; trong đồ thị có hướng, ta phân biệt bậc vào và bậc ra.

Từ một đồ thị có hướng, có thể lấy phiên bản vô hướng bằng cách thay mỗi cạnh $[v, w]$ bằng $\{v, w\}$ và xóa cạnh trùng. Ngược lại, phiên bản có hướng của đồ thị vô hướng thay mỗi cạnh $\{v, w\}$ bằng hai cạnh $[v, w]$ và $[w, v]$. Nếu $G_1 = [V_1, E_1]$ và $G_2 = [V_2, E_2]$ cùng loại, G_1 là đồ thị con của G_2 khi $V_1 \subseteq V_2$ và $E_1 \subseteq E_2$; nó là đồ thị con phủ nếu $V_1 = V_2$. Đồ thị con cảm sinh bởi tập đỉnh giữ mọi cạnh của đồ thị gốc có cả hai đầu trong tập đó; đồ thị con cảm sinh bởi tập cạnh giữ đúng các đầu của những cạnh được chọn.

Một đường đi từ v_1 tới v_k là danh sách đỉnh $[v_1, v_2, \dots, v_k]$ sao cho (v_i, v_{i+1}) là cạnh với mọi $i \in [1..k-1]$. Đường đi đơn nếu các đỉnh phân biệt. Trong đồ thị có hướng, chu trình là đường đi có $k > 1$ và $v_1 = v_k$; chu trình đơn còn yêu cầu các đỉnh $v_1, \dots, v_{k-1}$ phân biệt. Trong đồ thị vô hướng, chu trình cũng yêu cầu không lặp cạnh. Đồ thị không có chu trình gọi là phi chu trình. Nếu có đường đi từ v tới w , ta nói w đạt được từ v .

Đồ thị vô hướng liên thông nếu mọi đỉnh đạt được từ mọi đỉnh khác; các đồ thị con liên thông cực đại là các thành phần liên thông và chúng phân hoạch tập đỉnh. Với đồ thị có hướng, thành phần liên thông trong nghĩa này được lấy từ phiên bản vô hướng của nó. Khi phân tích thuật toán đồ thị, sách dùng n cho số đỉnh và m cho số cạnh. Đồ thị dày nếu m lớn so với n , thưa trong trường hợp ngược lại; nghĩa chính xác phụ thuộc ngữ cảnh. Thường giả sử $m = \Omega(n)$, vì nếu đồ thị quá ít cạnh thì có thể xử lý từng thành phần liên thông riêng.

Biểu diễn đồ thị thường dùng tập đỉnh và, với mỗi đỉnh, một hoặc hai tập cạnh liên thuộc. Với đồ thị có hướng, ta dùng $\text{out}(v) = \{[v, w] \in E\}$ và khi cần $\text{in}(v) = \{[u, v] \in E\}$. Với đồ thị vô hướng, ta dùng tập cạnh $\text{edges}(v)$ liên thuộc với v , hoặc biểu diễn phiên bản có hướng của đồ thị. Ma trận kề $n \times n$ cũng có thể dùng, với $A(v, w) = \text{true}$ nếu (v, w) là cạnh, nhưng nó tốn $\Omega(n^2)$ bộ nhớ và thường ép các thuật toán không tầm thường tốn $\Omega(n^2)$ thời gian, quá đắt cho đồ thị thưa.

Một cây tự do là đồ thị vô hướng liên thông và phi chu trình. Cây tự do n đỉnh có $n - 1$ cạnh và giữa hai đỉnh bất kỳ có đúng một đường đi đơn. Cây có gốc là cây tự do với một đỉnh đặc biệt r . Nếu v nằm trên đường từ r tới w , v là tổ tiên của w và w là hậu duệ của v . Nếu thêm $v \neq w$, đó là tổ tiên/hậu duệ thật sự. Nếu hai đỉnh kề nhau và một đỉnh là tổ tiên thật sự của đỉnh kia, ta có quan hệ cha-con. Mọi đỉnh trừ gốc có đúng một cha $p(v)$; đỉnh không có con là lá.

Độ sâu của đỉnh v được định nghĩa đệ quy: gốc có độ sâu 0, còn $\text{depth}(v) = \text{depth}(p(v)) + 1$. Chiều cao của đỉnh là 0 nếu nó là lá; nếu không, đó là 1 cộng chiều cao lớn nhất của một con. Cây con gốc tại v gồm các hậu duệ của v với gốc v . Tổ tiên chung gần nhất của hai đỉnh là đỉnh sâu nhất đồng thời là tổ tiên của cả hai.

Duyệt cây là thăm mỗi đỉnh của cây có gốc đúng một lần. Trong thứ tự trước (*preorder*), cha được thăm trước con; trong thứ tự sau (*postorder*), con được thăm trước cha. Duyệt theo chiều rộng thăm gốc, rồi lần lượt thăm các con chưa thăm của đỉnh được thăm sớm nhất còn con chưa thăm. Cách duyệt này cài bằng hàng đợi. Nếu cây được biểu diễn bằng tập con của mỗi đỉnh, ba phép duyệt trên đều tốn $O(n)$.

Cây nhị phân đầy đủ là cây có gốc trong đó mỗi đỉnh hoặc có đúng hai con, trái và phải, hoặc không có con. Đỉnh có hai con là đỉnh trong; đỉnh không con là đỉnh ngoài. Bỏ các đỉnh ngoài khỏi cây nhị phân đầy đủ cho ta cây nhị phân, nơi mỗi nút có thể thiếu con trái, con phải hoặc cả hai. Ngoài *preorder* và *postorder*, cây nhị phân còn có thứ tự giữa (*inorder*): duyệt trái, thăm nút, rồi duyệt phải. Khi cây

được dùng làm cấu trúc dữ liệu, sách gọi các đỉnh là nút và thường nghĩ chúng là nút trong bộ nhớ của máy con trở.

Một rừng là một họ cây rời nhau theo đỉnh. Cây phủ của đồ thị G là đồ thị con phủ của G đồng thời là cây: cây tự do nếu G vô hướng, hoặc cây có gốc với cạnh hướng từ cha tới con nếu G có hướng.

Ý tưởng duyệt cây mở rộng thành tìm kiếm trên đồ thị. Bắt đầu từ đỉnh s , ta thăm s , rồi lặp bước chọn một cạnh chưa xét (v, w) với v đã thăm, xét cạnh đó, và thăm w nếu w chưa thăm. Quá trình này thăm đúng một lần mọi đỉnh đạt được từ s , xét đúng một lần các cạnh rời từ vùng đạt được, và tạo ra một cây phủ của đồ thị con cảm sinh bởi các đỉnh đạt được: cạnh (v, w) thuộc cây nếu chính việc xét nó làm w được thăm.

Thứ tự chọn cạnh quyết định kiểu tìm kiếm. Trong DFS, chọn cạnh có đầu v là đỉnh được thăm gần đây nhất; có thể cài bằng ngăn xếp hoặc đệ quy. Trong BFS, chọn cạnh có đầu v là đỉnh được thăm sớm nhất còn cạnh chưa xét; có thể cài bằng hàng đợi. Cả hai đều chạy trong $O(m)$ nếu đồ thị được biểu diễn bằng danh sách cạnh ra/kề.

DFS cho một cách tìm thứ tự topo của đồ thị có hướng phi chu trình. Nếu sắp đỉnh theo thứ tự giảm dần của thời điểm hậu thăm (*postvisit*) sau một DFS, và lặp DFS từ đỉnh chưa thăm mới cho tới khi hết đỉnh, ta thu được thứ tự topo trong $O(m)$. Tính đúng đắn dựa trên việc trong khi DFS chạy, các đỉnh trên ngăn đệ quy tạo thành một đường đi từ gốc tìm kiếm tới đỉnh hiện tại, và thứ tự hậu thăm đảo chiều mọi cạnh trong DAG theo đúng hướng topo.

BFS cho khoảng cách theo số cạnh từ đỉnh bắt đầu. Đặt $\text{level}(s) = 0$; khi xét cạnh $[v, w]$ tới một đỉnh chưa thăm, đặt $\text{level}(w) = \text{level}(v) + 1$. Khi đó mọi cạnh $[v, w]$ thỏa $\text{level}(w) \leq \text{level}(v) + 1$, và $\text{level}(v)$ chính là độ dài đường đi ngắn nhất từ s tới v , nếu độ dài đường đi được tính bằng số cạnh. Lý do là hàng đợi của BFS lấy đỉnh ra theo thứ tự không giảm của mức.

4 Chương 2: Các tập rời nhau

Bài toán tập rời nhau yêu cầu duy trì một phân hoạch của tập phần tử dưới các thao tác tìm đại diện và hợp hai tập. Cấu trúc cây nén, thường được biết đến qua *union-find*, là ví dụ kinh điển cho phân tích khẩu hao: một thao tác có thể tốn nhiều thời gian, nhưng trung bình trên cả dãy thao tác thì rất nhỏ.

Hai ý tưởng cơ bản là hợp theo hạng/kích thước và nén đường đi. Khi kết hợp, chúng cho thời gian gần tuyến tính, với hệ số nghịch đảo Ackermann rất chậm tăng. Đây là cấu trúc nền cho cây khung nhỏ nhất và nhiều thuật toán đồ thị khác.

Tập rời nhau và cây nén

Chương này bắt đầu bằng một thuật toán rất dễ cài đặt nhưng có phân tích thời gian đáng chú ý. Ta cần duy trì một họ tập rời nhau dưới ba thao tác:

- $\text{makeset}(x)$: tạo tập mới chỉ chứa phần tử x , trước đó chưa thuộc tập nào;
- $\text{find}(x)$: trả về phần tử chuẩn đại diện cho tập chứa x ;
- $\text{link}(x, y)$: hợp hai tập có phần tử chuẩn lần lượt là x và y , hủy hai tập cũ, rồi chọn và trả về phần tử chuẩn của tập mới.

Cấu trúc của Galler–Fischer biểu diễn mỗi tập bằng một cây có gốc. Các nút là các phần tử của tập, phần tử chuẩn là gốc. Mỗi nút x giữ con trỏ cha $p(x)$; gốc trỏ tới chính nó. Khi $\text{makeset}(x)$, đặt $p(x) = x$. Khi $\text{find}(x)$, đi theo các con trỏ cha cho tới gốc và trả về gốc. Khi $\text{link}(x, y)$, trong phiên bản ngây thơ có thể đặt $p(x) = y$ và chọn y làm chuẩn mới. Nếu chỉ làm vậy, một thao tác find có thể tốn $O(n)$.

Hai mẹo kinh điển làm thuật toán gần tuyến tính. Mẹo thứ nhất là nén đường đi: khi $\text{find}(x)$ đã tìm được gốc r , mọi nút trên đường từ x tới r được đổi cha trực tiếp thành r . Một lần find có thể tốn thêm một hằng số, nhưng các lần find sau rẻ hơn nhiều.

Mẹo thứ hai là hợp theo hạng. Mỗi nút x có một số nguyên không âm $\text{rank}(x)$, là cận trên cho chiều cao của x . Khi tạo tập mới, hạng bằng 0. Khi hợp hai gốc, gốc hạng nhỏ hơn trở tới gốc hạng lớn hơn. Nếu hai hạng bằng nhau, chọn một gốc làm cha và tăng hạng của nó lên một. Đây là biến thể của hợp theo kích thước: mục tiêu là ngăn cây phát triển thành một dây dài.

Với hai mẹo đó, thủ tục find có dạng đệ quy quen thuộc:

nếu $x \neq p(x)$ thì đặt $p(x) := \text{find}(p(x))$; trả về $p(x)$.

Thủ tục link so sánh hai hạng, có thể hoán đổi x, y để bảo đảm x có hạng không lớn hơn y , tăng hạng y khi hai hạng bằng nhau, rồi đặt $p(x) = y$.

Cận khẩu hao cho nén đường đi

Gọi m là số thao tác và n là số phần tử. Khi đó có n thao tác makeset , nhiều nhất $n - 1$ thao tác link , và $m \geq n$. Phân tích khó vì nén đường đi thay đổi hình dạng cây theo cách không cục bộ. Các cận lịch sử đi từ $O(m \log \log n)$, tới $O(m \log^* n)$, rồi tới cận đúng $O(m\alpha(m, n))$, trong đó α là một nghịch đảo của hàm Ackermann.

Ba tính chất hạng là nền của phân tích:

- với mọi nút x , $\text{rank}(x) \leq \text{rank}(p(x))$, và bất đẳng thức là nghiêm nếu $p(x) \neq x$;
- nếu gốc cây là x , cây đó có ít nhất $2^{\text{rank}(x)}$ nút;
- số nút có hạng đúng k nhiều nhất là $n/2^k$, nên mọi hạng đều không vượt quá $\lg n$.

Từ đó, nếu chỉ dùng hợp theo hạng, một đường find có hạng tăng nghiêm dọc theo đường lên gốc, nên một find đơn lẻ tốn $O(\log n)$. Cận tốt hơn đến từ khẩu hao: không tính riêng từng find theo độ dài hiện tại, mà phân bổ chi phí trên cả dãy thao tác.

Tarjan trình bày phân tích bằng tín dụng và ghi nợ. Một tín dụng trả cho một lượng tính toán hằng. Mỗi makeset và link nhận một tín dụng. Mỗi find nhận $\alpha(m, n) + 2$ tín dụng. Nếu đường find dài hơn số tín dụng trực tiếp, ta tạo các cặp tín dụng-ghi nợ để trả trước cho thao tác hiện tại, rồi chứng minh tổng số ghi nợ còn lại sau cả dãy thao tác bị chặn bởi $O(m\alpha(m, n))$.

Ý tưởng kỹ thuật là chia các hạng $0, \dots, \lg n$ thành nhiều mức khối dựa trên hàm Ackermann. Mức càng cao thì phân hoạch càng thô. Mức của một nút là mức nhỏ nhất tại đó hạng của nút và hạng của cha nó nằm trong cùng một khối. Khi nén một đường find , nếu một nút không phải nút cuối cùng ở mức của nó trên đường, nút đó nhận một ghi nợ; nhưng mỗi ghi nợ như vậy buộc hạng của cha nó nhảy sang một khối thấp hơn mới. Vì số lần nhảy khối có thể đếm được theo các phân hoạch Ackermann, tổng ghi nợ bị chặn và thu được định lý:

union-find với nén đường đi và hợp theo hạng chạy trong $O(m\alpha(m, n))$ thời gian.

Trong mọi kích thước thực tế, $\alpha(m, n)$ không vượt quá một hằng số rất nhỏ; vì vậy cấu trúc này thường được xem là gần tuyến tính.

Biến thể và ghi chú

Nén đường đi đầy đủ có nhược điểm thực dụng là thường cần hai lượt trên đường find : một lượt tìm gốc và một lượt đổi cha. Có các biến thể một lượt vẫn đạt cận $O(m\alpha(m, n))$ khi kết hợp với hợp theo hạng.

Biến thể đáng chú ý là *path halving*: khi đi lên đường *find*, cứ mỗi nút khác lại cho nó trở tới ông của nó. Cách này đơn giản, dùng ít thao tác hơn trong cài đặt, và vẫn giữ tinh thần làm phẳng cây dần dần.

Tarjan cũng chứng minh cận dưới $\Omega(m\alpha(m, n))$ cho một lớp rộng các phương pháp thao tác con trở duy trì tập rời nhau, nên cận trên ở trên là chặt trong mô hình đó. Tuy vậy, nếu biết trước mẫu các thao tác link, Gabow và Tarjan có thuật toán tuyến tính cho một trường hợp đặc biệt bằng cách kết hợp nén đường đi trên tập lớn và tra bảng trên tập nhỏ. Nén đường đi còn xuất hiện ngoài union-find, trong các bài toán duy trì hàm trên đường đi của cây.

5 Chương 3: Heap

Heap là cấu trúc dữ liệu cho hàng đợi ưu tiên. Sách bàn về cây có thứ tự heap, các biến thể đơn giản và heap trái. Trong tối ưu mạng, hàng đợi ưu tiên xuất hiện tự nhiên trong Prim, Dijkstra và nhiều thuật toán gần nhãn.

Điểm quan trọng không chỉ là biết một heap chuẩn, mà là chọn biến thể phù hợp với kiểu thao tác cần hỗ trợ: chèn, lấy phần tử nhỏ nhất, giảm khóa, và ghép hai heap. Heap trái là một ví dụ cho việc thay đổi hình dạng cây để thao tác ghép được nhanh.

Heap và cây có thứ tự heap

Heap là cấu trúc trừu tượng gồm một tập phần tử, mỗi phần tử có một khóa thực. Hai thao tác cơ bản là $\text{insert}(i, h)$, chèn phần tử i vào heap h , và $\text{deletemin}(h)$, xóa rồi trả về một phần tử có khóa nhỏ nhất. Thao tác $\text{makeheap}(s)$ tạo heap từ một tập phần tử. Tùy ứng dụng, ta còn dùng findmin để xem phần tử nhỏ nhất mà không xóa, delete để xóa phần tử bất kỳ, và meld để ghép hai heap rời nhau.

Một cách cài heap là dùng cây có thứ tự heap: mỗi nút chứa một phần tử, và khóa của cha không lớn hơn khóa của con. Vì vậy gốc chứa một phần tử khóa nhỏ nhất, nên findmin chỉ cần truy cập gốc và tốn $O(1)$. Thời gian các thao tác còn lại phụ thuộc vào cách khống chế hình dạng cây.

d -heap

Với heap ngoại sinh, phần tử và nút cây là hai đối tượng khác nhau; khi cập nhật, ta có thể phục hồi thứ tự heap bằng cách di chuyển phần tử giữa các nút. Để chèn i , thêm một lá rỗng x . Nếu cha của x có khóa lớn hơn $\text{key}(i)$, đẩy phần tử ở cha xuống x , chuyển ô rỗng lên cha, và lặp lại. Khi dừng, đặt i vào ô rỗng. Đây là thao tác *sift-up*.

Để xóa một phần tử i , lấy phần tử j ở lá được tạo sau cùng chưa bị xóa, xóa lá đó, rồi dùng j lấp vào vị trí của i . Nếu khóa của j nhỏ hơn hoặc bằng khóa cũ của i , phục hồi bằng *sift-up*. Nếu lớn hơn, dùng *sift-down*: so sánh j với các con của ô rỗng, đưa con có khóa nhỏ nhất lên nếu nó nhỏ hơn j , rồi tiếp tục đi xuống cho tới khi hợp lệ.

Hình dạng cây được chọn là cây d -phân đầy đủ: mỗi nút có nhiều nhất d con và nút được thêm theo thứ tự duyệt rộng. Một d -heap là cây d -phân đầy đủ có một phần tử mỗi nút và thỏa thứ tự heap. Với n phần tử, findmin tốn $O(1)$, chèn tốn $O(\log_d n)$, còn delete và deletemin tốn $O(d \log_d n)$. Tham số d cho phép điều chỉnh theo tần suất thao tác: nếu xóa ít hơn chèn, có thể chọn d lớn hơn để giảm chiều sâu.

Vì cây d -phân đầy đủ rất đều, không cần lưu con trỏ. Đánh số các nút từ 1 tới n theo thứ tự duyệt rộng, cha của nút x là $\lfloor (x-1)/d \rfloor$, còn các con của x nằm trong khoảng

$$[d(x-1) + 2 .. \min\{dx + 1, n\}].$$

Do đó heap được biểu diễn bằng một mảng/ánh xạ từ vị trí 1.. n tới phần tử. Nếu cần xóa phần tử bất kỳ, mỗi phần tử cũng nên giữ chỉ số heap ngược lại để biết vị trí hiện tại của nó.

Thao tác makeheap không nên thực hiện bằng n lần chèn, vì tốn $O(n \log_d n)$. Cách tốt hơn là đặt các phần tử tùy ý vào cây d -phân đầy đủ, rồi chạy sift-down từ các nút $n, n-1, \dots, 1$. Vì số nút ở chiều cao i giảm theo d^i , tổng thời gian là tuyến tính $O(n)$. Đây là dạng tổng quát của thuật toán xây heap từ dưới lên.

Về lịch sử, heap nhị phân xuất phát từ thuật toán heapsort của Williams, dựa trên ý tưởng *Treesort* 3 trước đó của Floyd; cách lưu cây đầy đủ bằng mảng cũng đi cùng truyền thống này. Johnson đề xuất tổng quát hóa sang $d > 2$. Tarjan lưu ý rằng nếu xét hằng số của các thao tác heap, lựa chọn $d = 3$ hoặc $d = 4$ thường có vẻ trội hơn $d = 2$, dù nhận xét này cần được kiểm chứng bằng thực nghiệm trên máy cụ thể. Điều quan trọng là d không phải một chi tiết trang trí: nó là tham số để cân bằng chi phí sift-up và sift-down theo tần suất chèn/xóa của thuật toán phía trên.

Heap trái

d -heap không thuận tiện cho meld. Khi cần ghép heap thường xuyên, Tarjan trình bày heap trái. Trong cây nhị phân đầy đủ, hạng của nút x là độ dài ngắn nhất từ x tới một nút ngoài. Cây là *leftist* nếu với mọi nút trong x , hạng của con trái ít nhất bằng hạng của con phải. Khi đó đường đi theo con phải từ gốc là một đường ngắn nhất tới nút ngoài, và có độ dài không quá $\lg n$.

Heap trái là cây leftist chứa một phần tử ở mỗi nút trong, các phần tử vẫn thỏa thứ tự heap. Nó thường được xem là cấu trúc nội sinh: chính phần tử là nút cây; nút ngoài được biểu diễn bằng null. Mỗi nút cần hai con trái/phải và một hạng, còn heap được nhận diện bằng con trở gốc.

Thao tác chính là meld. Để ghép hai heap, trộn hai đường phải theo thứ tự khóa không giảm, rồi đi ngược lại cập nhật hạng và đổi hai con khi cần để khôi phục tính leftist. Vì mỗi đường phải dài $O(\log n)$, meld tốn $O(\log n)$. Chèn một phần tử là tạo heap một nút rồi meld; xóa phần tử nhỏ nhất là bỏ gốc rồi meld hai cây con trái/phải. Cả hai thao tác đều tốn $O(\log n)$.

Thứ tự heap cũng cho phép liệt kê mọi phần tử có khóa không vượt quá một ngưỡng x . Ta duyệt preorder từ gốc: gặp nút có khóa lớn hơn x thì dừng cả cây con đó; gặp nút hợp lệ thì đưa nó vào kết quả và duyệt hai con. Với heap trái, thời gian tỉ lệ với số phần tử được liệt kê.

Thao tác heapify cho một danh sách heap rời nhau có thể làm như merge sort: dùng danh sách làm hàng đợi, lặp bước lấy hai heap đầu, meld chúng, rồi đưa heap mới về cuối. Với k heap ban đầu và tổng n phần tử, phân tích từng lượt qua hàng đợi cho cận $O(k \max\{1, \log(n/k)\})$. Đặc biệt, nếu bắt đầu từ n heap một nút, ta tạo heap trái từ n phần tử trong $O(n)$.

Xóa tùy ý trong heap trái có thể đạt $O(\log n)$ nếu thêm con trở cha, nhưng Chương 3 nêu một cách phù hợp hơn cho các ứng dụng sau: xóa lười. Khi muốn xóa, chỉ đánh dấu phần tử là đã xóa. Đến findmin hoặc deletemin, duyệt cây để lấy các nút chưa xóa mà mọi tổ tiên thật sự của chúng đã bị đánh dấu, rồi heapify các cây con tương ứng. Meld cũng có thể làm lười bằng cách tạo nút giả có hai con là hai heap cần ghép; khi tìm min, nút giả được xử lý như nút đã xóa. Cách này làm meld và delete chỉ tốn $O(1)$ tại thời điểm gọi, còn chi phí thật được trả dồn trong lần tìm/xóa min sau.

Nếu một lần findmin loại bỏ k nút giả hoặc nút đã đánh dấu khỏi một heap có n mục thật sự liên quan, thời gian của bước purge rồi heapify là

$$O\left(k \max\left\{1, \log \frac{n}{k+1}\right\}\right).$$

Vì vậy xóa lười đặc biệt hữu ích khi có thể nhận ra các mục đã chết bằng một dấu hiệu ngầm, không cần đi tới từng heap để xóa ngay. Chương 6 dùng chính ý tưởng này trong thuật toán cây khung nhỏ nhất: cạnh đã trở thành nội bộ của một thành phần có thể bị loại khi nó nổi lên ở đỉnh heap, thay vì bị xóa khỏi mọi heap liên quan tại thời điểm hai thành phần được ghép.

Một mở rộng hữu ích là addtokeys(x, h), cộng x vào khóa của mọi phần tử trong heap h . Thay vì lưu khóa tuyệt đối ở mỗi nút, lưu hiệu khóa giữa nút và cha; khóa thật của một nút là tổng các hiệu trên

đường tới gốc. Khi đó cộng cùng một lượng vào mọi khóa chỉ cần cộng vào hiệu khóa ở gốc trong $O(1)$, còn bậc thời gian của các thao tác heap khác không đổi.

Tarjan cũng nhấn mạnh rằng heap trái không phải cách duy nhất để biểu diễn heap có meld. Gần như mọi họ cây cân bằng cho phép ghép nhanh hai cây đang thỏa thứ tự heap đều có thể dùng được. Bằng chứng thực nghiệm và lý thuyết của Brown cho thấy heap nhị thức thường có hằng số tốt hơn khi quan tâm hiệu năng thực tế. Đổi lại, các thao tác trên heap nhị thức khó mô tả và khó cài đặt hơn heap trái, nên heap trái vẫn là lựa chọn sự phạm tốt trong sách: đủ mạnh cho các thuật toán mạng phía sau, nhưng vẫn giữ được cấu trúc đơn giản.

6 Chương 4: Cây tìm kiếm

Chương này nhắc lại cây tìm kiếm nhị phân cho tập có thứ tự, rồi chuyển sang cây cân bằng và cây tự điều chỉnh. Cây cân bằng giữ chiều cao $O(\log n)$ để bảo đảm tìm kiếm, chèn và xóa hiệu quả. Cây tự điều chỉnh, tiêu biểu là splay tree của Sleator và Tarjan, không lưu thông tin cân bằng tường minh mà dựa vào các phép xoay sau mỗi truy cập.

Ý nghĩa trong sách là chuẩn bị cho những cấu trúc động phức tạp hơn, đặc biệt là biểu diễn đường bằng cây nhị phân ở chương kế tiếp.

Tập có thứ tự và cây tìm kiếm nhị phân

Chương này xét bài toán duy trì một hoặc nhiều tập phần tử có khóa phân biệt trong một miền có thứ tự toàn phần. Các thao tác cơ bản là $\text{access}(k, s)$, trả về phần tử trong s có khóa k nếu có; $\text{insert}(i, s)$, chèn i ; và $\text{delete}(i, s)$, xóa i . Thao tác makesortedset tạo tập rỗng. Hai thao tác mạnh hơn là $\text{join}(s_1, i, s_2)$, ghép s_1 , phần tử i , và s_2 khi mọi khóa trong s_1 nhỏ hơn $\text{key}(i)$ và mọi khóa trong s_2 lớn hơn; và $\text{split}(i, s)$, tách s thành phần nhỏ hơn i , riêng i , và phần lớn hơn i .

Biểu diễn chuẩn là cây tìm kiếm nhị phân: mỗi nút trong chứa một phần tử; mọi khóa trong cây con trái của x nhỏ hơn khóa ở x , mọi khóa trong cây con phải lớn hơn. Cây được xem là nội sinh, phần tử chính là nút; nút ngoài rỗng được biểu diễn bằng null. Mỗi nút thường giữ con trái, con phải và cha. Tìm kiếm khóa k bắt đầu từ gốc, đi trái nếu khóa hiện tại lớn hơn k , đi phải nếu nhỏ hơn k , và dừng khi gặp khóa cần tìm hoặc null.

Chèn tương tự tìm kiếm, nhưng giữ thêm con trở theo sau tới cha của vị trí rỗng sẽ được thay bằng nút mới. Xóa phức tạp hơn: nếu nút cần xóa có con rỗng, thay nó bằng con không rỗng nếu có. Nếu có đủ hai con, hoán đổi nó với tiền nhiệm theo thứ tự giữa, tức nút cực phải trong cây con trái; tiền nhiệm này có con phải rỗng, nên sau hoán đổi có thể xóa bằng trường hợp đơn giản.

Join của hai cây tìm kiếm với phần tử chèn giữa rất dễ: đặt gốc của s_1 làm con trái của i , gốc của s_2 làm con phải, rồi trả về i . Split tại i đi dọc đường từ i lên gốc, cắt các cạnh trên đường đó và dùng một dây join để gom các cây con chứa khóa nhỏ hơn i vào cây trái, các cây con chứa khóa lớn hơn vào cây phải. Nếu không kiểm soát hình dạng, cây tìm kiếm có thể trở thành một đường dài n , nên mọi thao tác trừ join có thể tốn $O(n)$.

Cây nhị phân cân bằng

Cách cổ điển để bảo đảm thời gian xấu nhất $O(\log n)$ là áp đặt điều kiện cân bằng khiến độ sâu của cây n nút là $O(\log n)$, rồi tái cân bằng sau hoặc trong mỗi cập nhật. Tarjan dùng một dạng cây nhị phân cân bằng tương đương với cây đỏ-đen. Mỗi nút x có một hạng nguyên $\text{rank}(x)$. Các hạng thỏa:

- nếu x có cha, thì $\text{rank}(x) \leq \text{rank}(p(x)) \leq \text{rank}(x) + 1$;

- nếu x có ông, thì $\text{rank}(x) < \text{rank}(p^2(x))$;
- nút ngoài có hạng 0, và cha của nút ngoài có hạng 1.

Gọi nút x là đen nếu cha của nó có hạng lớn hơn hạng của nó đúng 1, hoặc nếu x không có cha; gọi là đỏ nếu cha và x cùng hạng. Khi đó mọi nút đỏ có cha đen, mọi nút ngoài đen, và mọi đường từ một nút tới nút ngoài có cùng số nút đen. Đây chính là cách nhìn đỏ-đen. Nếu co mỗi nút đỏ vào cha của nó, ta nhận được cây 2, 4: mỗi nút trong có 2, 3, hoặc 4 con và mọi nút ngoài có cùng độ sâu.

Một nút hạng k có chiều cao nhiều nhất $2k$ và có ít nhất $2^{k+1} - 1$ hậu duệ. Vì vậy cây nhị phân cân bằng có n nút trong có độ sâu nhiều nhất $2\lceil \lg(n+1) \rceil$, nên truy cập mất $O(\log n)$. Sau chèn hoặc xóa, cây được sửa bằng các thao tác hằng thời gian: tăng hạng (*promotion*), giảm hạng (*demotion*), xoay đơn và xoay kép.

Sau chèn, vi phạm duy nhất có thể là một nút đỏ có cha đỏ. Nếu ông của nút hiện tại có hai con đỏ, tăng hạng ông và tiếp tục kiểm tra lên trên. Nếu không, một xoay đơn hoặc xoay kép thích hợp kết thúc việc sửa. Do đó chèn cần một dãy promotion và nhiều nhất hai xoay đơn, tổng $O(\log n)$.

Sau xóa, một nút có thể có hạng nhỏ hơn cha của nó tới hai đơn vị. Nếu anh em của nút là đen và cả hai con của anh em đen, giảm hạng cha rồi tiếp tục lên. Nếu anh em đen nhưng có con đỏ phù hợp, dùng xoay đơn hoặc xoay kép để kết thúc. Nếu anh em đỏ, xoay một lần để chuyển về trường hợp anh em đen. Tổng cộng xóa cần một dãy demotion và nhiều nhất ba xoay đơn, vẫn $O(\log n)$.

Join hai cây cân bằng được thực hiện bằng cách so sánh hạng hai gốc. Nếu $\text{rank}(s_1) \geq \text{rank}(s_2)$, đi theo con phải của s_1 cho tới nút có hạng bằng hạng của s_2 , chèn phần tử i tại đó với cây con trái là phần vừa tách và cây con phải là s_2 , rồi tái cân bằng như sau chèn. Trường hợp đối xứng xử lý tương tự. Split dùng dãy join như cây tìm kiếm thường; các chi phí join tạo thành tổng lồng nhau và cộng lại $O(\log n)$.

Cây nhị phân tự điều chỉnh

Một hướng khác là bỏ điều kiện cân bằng tường minh và chấp nhận cận khẩu hao. Sleator và Tarjan đề xuất thao tác *splay*. Để *splay* tại nút x , đi từ x lên gốc và xoay theo từng cặp:

- nếu x có cha nhưng không có ông, xoay tại cha;
- nếu x và cha của nó đều là con trái hoặc đều là con phải, xoay tại ông rồi xoay tại cha;
- nếu một bên trái một bên phải, xoay tại cha rồi xoay tại cha mới của x .

Kết quả là x trở thành gốc, còn các nút trên đường tìm kiếm được sắp lại.

Cây tìm kiếm tự điều chỉnh thực hiện *splay* trong mỗi thao tác trừ join. Sau khi truy cập hoặc chèn i , *splay* tại i . Sau khi xóa i , *splay* tại cha của nó ngay trước khi xóa. Trước khi split tại i , *splay* tại i ; khi đó split chỉ còn trả về hai cây con trái/phải của gốc. Thời gian thao tác tỉ lệ với độ dài đường *splay*.

Phân tích dùng thế năng bằng tín dụng. Gán mỗi phần tử một trọng số dương; tổng trọng số của nút x là tổng trọng số các hậu duệ của x , và $\text{rank}(x) = \lceil \lg \text{tw}(x) \rceil$. Duy trì bất biến rằng mỗi nút trong giữ số tín dụng bằng hạng của nó. Một lần *splay* tại nút x trong cây có gốc v cần nhiều nhất

$$3(\text{rank}(v) - \text{rank}(x)) + 1$$

tín dụng để trả cho các bước xoay và khôi phục bất biến. Nếu chọn mọi trọng số bằng 1, mỗi hạng nằm trong $[0.. \lceil \lg n \rceil]$, nên *splay* tốn $O(\log n)$ khẩu hao.

Từ đó, một dãy m thao tác trên tập có thứ tự, bắt đầu từ rỗng và có n thao tác chèn/join tạo phần tử, tốn tổng $O(m \log n)$ thời gian. Với trọng số khác nhau, cây *splay* còn đạt hiệu quả khẩu hao so với cây tìm kiếm tĩnh tối ưu đến hằng số. Điểm quan trọng cho chương sau là cây tự điều chỉnh đôi khi cho thuật toán tiệm cận tốt hơn cách dùng cây cân bằng thông thường.

7 Chương 5: Nối và cắt cây

Bài toán *link-cut tree* yêu cầu duy trì một rừng động dưới các thao tác nối hai cây, cắt một cạnh, và truy vấn/cập nhật trên đường trong cây. Tarjan trình bày ý tưởng biểu diễn mỗi cây bằng một tập các đường, rồi biểu diễn mỗi đường bằng một cây nhị phân phụ.

Cấu trúc này là một trong các đóng góp nổi tiếng của Sleator–Tarjan. Nó cho phép đưa nhiều thuật toán đồ thị động hoặc thuật toán tối ưu mạng từ mức ý tưởng xuống mức cài đặt hiệu quả.

Bài toán nối và cắt cây

Các cây ở những chương trước chủ yếu là biểu diễn cụ thể của cấu trúc dữ liệu trừu tượng. Ở đây cây xuất hiện trực tiếp hơn: ta cần duy trì một họ cây có gốc, rời nhau theo đỉnh, trong khi cạnh được thêm hoặc xóa theo thời gian. Mỗi đỉnh có một chi phí thực. Các thao tác là:

- $\text{maketree}(v)$: tạo cây một đỉnh v , chi phí 0;
- $\text{findroot}(v)$: trả về gốc của cây chứa v ;
- $\text{findcost}(v)$: trên đường từ v tới gốc, trả về đỉnh gần gốc nhất có chi phí nhỏ nhất và giá trị chi phí đó;
- $\text{addcost}(v, x)$: cộng x vào chi phí mọi đỉnh trên đường từ v tới gốc;
- $\text{link}(v, w)$: thêm cạnh có hướng từ gốc v sang đỉnh w của cây khác;
- $\text{cut}(v)$: xóa cạnh đi ra khỏi v , với v không phải gốc.

Nếu chỉ lưu cha và chi phí ở mỗi đỉnh, các thao tác $\text{maketree}/\text{link}/\text{cut}$ là $O(1)$, nhưng $\text{findroot}/\text{findcost}/\text{addcost}$ có thể tốn theo độ sâu, tức $O(n)$. Mục tiêu của chương là biểu diễn cây ẩn để các thao tác đường đi tốn $O(\log n)$ khâu hao, còn link/cut cũng chỉ $O(\log n)$.

Biểu diễn cây bằng các đường

Giả sử ta đã có cấu trúc hỗ trợ các thao tác trên một họ đường rời nhau: makepath , findpath , findtail , findpathcost , addpathcost , join , và split . Ta dùng các đường này để biểu diễn cây bằng cách tô mỗi cạnh là đặc hoặc đứt, với điều kiện nhiều nhất một cạnh đặc đi vào mỗi đỉnh. Các cạnh đặc phân hoạch cây thành những đường đặc rời nhau.

Với mỗi đường đặc p , lưu $\text{successor}(p)$, là đỉnh mà cạnh đứt rời khỏi đuôi của p đi vào; nếu đuôi của p là gốc cây thì successor là null. Thao tác tổng hợp quan trọng là $\text{expose}(v)$: biến toàn bộ đường từ v tới gốc thành một đường đặc bằng cách đổi một số cạnh đứt thành đặc và đẩy các cạnh đặc lệch khỏi đường đó thành đứt.

Trong expose , mỗi vòng lặp được gọi là một *splice*. Một splice tách đường đặc hiện tại tại v , cập nhật successor cho phần bị tách ra, rồi nối phần đã lộ phía dưới với v và đoạn phía trên. Mỗi thao tác cây dùng $O(1)$ thao tác đường và nhiều nhất hai expose . Vì vậy phần còn lại là đếm số splice.

Đặt $\text{size}(v)$ là số hậu duệ của v , kể cả v . Cạnh từ v tới cha w là nặng nếu $2 \text{size}(v) > \text{size}(w)$, và nhẹ nếu không. Mỗi đỉnh có nhiều nhất một cạnh nặng đi vào, và trên mọi đường từ một đỉnh tới gốc có nhiều nhất $\lceil \lg n \rceil$ cạnh nhẹ.

Phân tích số splice theo số cạnh vừa đặc vừa nặng. Ban đầu không có cạnh như vậy, cuối cùng nhiều nhất $n - 1$. Trong một expose , mỗi splice trừ splice đầu đổi một cạnh đứt thành đặc; chỉ $O(\log n)$ trong số đó có thể là cạnh nhẹ. Mỗi lần đổi một cạnh nặng đứt thành đặc làm tăng số cạnh nặng–đặc, trừ khi

sau này nó bị phá bởi expose, link hoặc cut. Đếm các lần tạo và phá cạnh nặng-đặc cho thấy m thao tác cây gây $O(m \log n)$ splice tổng cộng, tức $O(\log n)$ splice khấu hao cho mỗi expose.

Biểu diễn đường bằng cây nhị phân

Mỗi đường đặc được biểu diễn bằng một cây nhị phân, trong đó thứ tự giữa của các nút chính là thứ tự đỉnh trên đường. Tarjan gọi các cây này là cây đặc (*solid tree*) để phân biệt với cây thật đang được duy trì. Gốc của cây đặc nhận diện đường đặc và giữ successor của đường.

Để hỗ trợ chi phí, mỗi nút x giữ hai giá trị hiệu thay vì chi phí tuyệt đối: $\Delta \text{cost}(x)$ và $\Delta \text{min}(x)$. Ý tưởng giống hiệu khóa ở heap: từ các hiệu dọc đường trong cây đặc, ta khôi phục được chi phí của đỉnh và chi phí nhỏ nhất trong cây con. Nhờ cách lưu chênh lệch này, xoay đơn/xoay kép, assemble và disassemble của cây đặc đều làm được trong $O(1)$ trong khi vẫn giữ đúng thông tin min.

Các thao tác đường được quy về thao tác cây tìm kiếm: makepath tạo cây một nút; findpath đi theo cha trong cây đặc tới gốc; findtail đi theo con phải tới cuối đường; findpathcost đi xuống theo thông tin min để tìm đỉnh cuối cùng có chi phí nhỏ nhất; addpathcost chỉ cộng vào giá trị min ở gốc đường; join và split là join/split của cây tìm kiếm.

Nếu dùng cây nhị phân cân bằng để biểu diễn đường đặc, mỗi thao tác đường tốn $O(\log n)$. Kết hợp với định lý số splice cho ta $O(m(\log n)^2)$ cho m thao tác cây. Có thể tiết kiệm trường nhờ bằng cách dùng chung một trường cho parent của nút không phải gốc cây đặc và successor của gốc cây đặc, kèm một bit cho biết nút có là gốc cây đặc hay không.

Dùng cây tự điều chỉnh

Cận được cải thiện còn $O(m \log n)$ nếu biểu diễn đường đặc bằng cây splay thay cho cây cân bằng. Khi findpath(v), ta splay v để nó trở thành gốc cây đặc. findtail đi sang phải tới đuôi rồi splay đỉnh đó. findpathcost đi xuống theo thông tin min rồi splay đỉnh tìm được. split(v) chỉ cần splay v rồi tách hai cây con trái/phải.

Phân tích dùng trọng số riêng của đỉnh v là

$$\text{size}(v) - \sum \text{size}(w),$$

trong đó tổng lấy trên các con đặc của v trong cây thật. Tổng trọng số của một nút trong cây đặc bằng số hậu duệ của nó trong cây ảo tương ứng. Vì giá trị này nằm giữa 1 và n , hạng theo $\lceil \lg tw \rceil$ nằm trong $O(\log n)$. Một splice cần $O(1)$ cộng với phần thể năng splay; các phần thể năng cộng dồn qua expose chỉ còn $O(\log n)$. Link cần thêm $O(\log n)$ tín dụng do tăng tổng trọng số của một nút; cut chỉ giảm trọng số nên giải phóng tín dụng. Vì vậy m thao tác cây tốn tổng $O(m \log n)$.

Chương cũng nêu các mở rộng. Quan trọng nhất là evert(v), đổi gốc cây chứa v để v trở thành gốc. Với evert, cấu trúc xử lý được cả cây tự do không gốc bằng cách gốc hóa lại khi cần. Có thể gán chi phí cho cạnh thay vì đỉnh. Nếu cần cận xấu nhất thay vì khấu hao, có biến thể dùng biased search tree, phức tạp hơn nhưng đạt $O(\log n)$ xấu nhất cho mỗi thao tác.

8 Chương 6: Cây khung nhỏ nhất

Chương này bắt đầu phần thuật toán mạng. Bài toán cây khung nhỏ nhất là môi trường tự nhiên để thấy phương pháp tham lam hoạt động chính xác. Các thuật toán cổ điển gồm Kruskal, Prim và Boruvka. Chúng khác nhau chủ yếu ở cách chọn cạnh an toàn tiếp theo và cấu trúc dữ liệu hỗ trợ.

Union-find làm Kruskal hiệu quả; heap hỗ trợ Prim; còn các biến thể như thuật toán vòng tròn khai thác cách tổ chức cạnh và thành phần liên thông để cải thiện cận thời gian trong một số mô hình.

Phương pháp tham lam

Nửa sau của sách dùng các cấu trúc dữ liệu đã xây để giải các bài toán tối ưu mạng. Với cây khung nhỏ nhất, đầu vào là đồ thị vô hướng liên thông có chi phí thực trên cạnh; cần tìm cây khung có tổng chi phí nhỏ nhất. Tarjan trình bày một khung tham lam bằng tô màu cạnh. Ban đầu mọi cạnh chưa tô. Cạnh được chấp nhận tô xanh, cạnh bị loại tô đỏ. Bất biến là luôn tồn tại một cây khung nhỏ nhất chứa mọi cạnh xanh và không chứa cạnh đỏ nào.

Hai quy tắc duy trì bất biến:

- Quy tắc xanh: chọn một lát cắt không bị cạnh xanh nào cắt qua; trong các cạnh chưa tô cắt qua nó, tô xanh một cạnh chi phí nhỏ nhất.
- Quy tắc đỏ: chọn một chu trình đơn không chứa cạnh đỏ; trong các cạnh chưa tô trên chu trình, tô đỏ một cạnh chi phí lớn nhất.

Quy tắc xanh là tính chất cut của MST, quy tắc đỏ là tính chất cycle. Nếu quá trình chưa tô hết cạnh, một cạnh chưa tô hoặc nối hai cây xanh khác nhau, khi đó áp dụng được quy tắc xanh, hoặc nằm trong cùng một cây xanh, khi đó áp dụng được quy tắc đỏ. Vì vậy phương pháp tô hết cạnh và các cạnh xanh tạo thành MST.

Ba thuật toán cổ điển

Boruvka bắt đầu với n cây xanh một đỉnh. Trong mỗi vòng, với mỗi cây xanh T , chọn cạnh chi phí nhỏ nhất liên thuộc với T rồi tô xanh các cạnh được chọn. Nếu chi phí không phân biệt, cần phá hòa bằng số thứ tự cạnh hoặc đỉnh. Mỗi vòng ghép nhiều cây cùng lúc, nên cách nhìn này tự nhiên cho thuật toán song song và là nền cho thuật toán vòng tròn sau.

Kruskal xét các cạnh theo thứ tự chi phí không giảm. Nếu cạnh hiện tại có hai đầu trong cùng một cây xanh thì tô đỏ, nếu không thì tô xanh và ghép hai cây. Cài đặt gồm hai phần: sắp cạnh theo chi phí và dùng union-find để kiểm tra/ghép thành phần xanh. Sắp cạnh tốn $O(m \log n)$; nếu cạnh đã sắp hoặc sắp nhanh được bằng radix sort, phần union-find tốn $O(m \alpha(m, n))$.

Prim bắt đầu từ một đỉnh s và chỉ duy trì một cây xanh không tầm thường T . Mỗi bước chọn cạnh chi phí nhỏ nhất nối T với ngoài T . Với mỗi đỉnh biên v , lưu cạnh xanh nhạt tốt nhất nối v vào T ; heap chứa các đỉnh biên với khóa là chi phí cạnh xanh nhạt đó. Khi một đỉnh mới vào T , xét các cạnh đi ra từ nó và giảm khóa các đỉnh biên nếu tìm được cạnh tốt hơn. Dùng d -heap với $d = 2 + \lceil m/n \rceil$, thời gian là $O(m \log_{2+m/n} n)$, tốt cho đồ thị dày.

Thuật toán vòng tròn

Khung quy tắc xanh tổng quát hơn chọn tùy ý một cây xanh, tìm cạnh chi phí nhỏ nhất liên thuộc với nó và tô xanh. Kruskal chọn cây theo thứ tự do cạnh quyết định; Prim chỉ phát triển một cây; Boruvka phát triển đều mọi cây. Cheriton và Tarjan đưa ra một cách gần Boruvka nhưng dùng cấu trúc dữ liệu để đạt $O(m \log \log n)$ trên đồ thị thưa.

Mỗi cây xanh cần hai cấu trúc. Thứ nhất là union-find cho tập đỉnh của cây. Thứ hai là heap trái các cạnh có ít nhất một đầu trong cây; khóa là chi phí cạnh. Một cạnh được xem là đã xóa nếu hai đầu hiện thuộc cùng cây xanh. Ta không cần đánh dấu cạnh trực tiếp: khi hai cây xanh được hợp, các cạnh nội bộ mới sẽ tự bị loại trong lần findmin nhờ xóa lười.

Các cây xanh được đưa vào một hàng đợi. Mỗi bước lấy cây đầu hàng đợi, chạy findmin trên heap cạnh của nó để lấy cạnh tốt nhất, tô xanh cạnh đó, xóa cây còn lại ở đầu kia khỏi hàng đợi, hợp hai tập đỉnh bằng union-find, meld hai heap cạnh, rồi đưa cây mới về cuối hàng đợi. Vì vậy thuật toán được gọi

là *round robin*.

Phân tích chia tiến trình thành các lượt qua hàng đợi. Ở lượt j , mỗi cây xanh trong hàng đợi có ít nhất 2^j đỉnh, nên chỉ có nhiều nhất $\lceil \lg n \rceil$ lượt. Tổng kích thước các heap được xử lý trong một lượt bị chặn bởi $O(m + n)$, vì mỗi cạnh thật nằm trong nhiều nhất hai heap và các nút giả do meld lười chỉ tuyến tính. Với findmin trên heap trái có xóa lười, tách các lần purge thành nhỏ và lớn cho tổng $O(m \log \log n)$. Các thao tác union-find phát sinh trong quá trình cũng nằm trong cùng cận, nên thuật toán vòng tròn chạy $O(m \log \log n)$.

Thuật toán vòng tròn nhanh hơn tiệm cận các thuật toán cổ điển trên đồ thị thưa. Trên đồ thị dày, Prim thường tốt hơn một hệ số $\log \log n$. Có thể thêm bước co đồ thị theo từng khoảng: loại cạnh nội bộ cây xanh và giữ cạnh rẽ nhất giữa mỗi cặp cây xanh. Khi đó cận trở thành $O(m \log \log_{2+m/n} n)$, tiệm cận tốt trên mọi mật độ đồ thị đã xét trong sách.

Biến thể

Một thuật toán trực tuyến xử lý cạnh theo thứ tự bất kỳ: thêm cạnh mới vào tập xanh; nếu tạo chu trình, bỏ cạnh xanh đắt nhất trên chu trình. Dùng union-find và dynamic tree có evert, cách này chạy $O(m \log n)$.

Các bài toán liên quan gồm kiểm tra một cây khung đã cho có tối ưu không, phân tích độ nhạy của từng cạnh trong MST, và cập nhật MST khi xóa một cạnh cây; các bài toán này có thuật toán $O(m \alpha(m, n))$. Một số lớp đồ thị đóng dưới phép co cạnh và có $O(n)$ cạnh, như đồ thị phẳng, cho phép tìm MST tuyến tính. Với ràng buộc bậc, một đỉnh bị ràng buộc còn quy về MST thường; ràng buộc trên tập đỉnh độc lập liên quan giao matroid; ràng buộc mọi đỉnh trở thành NP-đầy đủ. Phiên bản có hướng là bài toán arborescence nhỏ nhất, có thuật toán $O(\min\{m \log n, n^2\})$.

9 Chương 7: Đường đi ngắn nhất

Chương này trình bày cây đường đi ngắn nhất, thuật toán gán nhãn và thao tác quét. Các thuật toán kinh điển như Dijkstra và Bellman–Ford có thể được nhìn chung qua cách chọn thứ tự quét đỉnh/cạnh. Khi trọng số không âm, hàng đợi ưu tiên cho phép chọn nhãn nhỏ nhất tiếp theo; khi có trọng số âm nhưng không có chu trình âm, cần quét lặp để lan truyền cải thiện.

Phần mọi cặp đỉnh nổi bài toán đơn nguồn với các phương pháp quy hoạch động và nhân ma trận theo đại số min-plus.

Cây đường đi ngắn nhất và gán nhãn

Đầu vào là đồ thị có hướng với độ dài thực trên cạnh, có thể âm. Độ dài đường đi là tổng độ dài cạnh. Chương phân biệt bốn phiên bản: một cặp nguồn–đích, một nguồn tới mọi đỉnh, mọi đỉnh tới một đích, và mọi cặp. Hai phiên bản một nguồn/một đích là đối ngẫu qua đảo hướng cạnh; các thuật toán một cặp thường giải ít nhất một phần bài toán một nguồn, nên chương tập trung vào một nguồn.

Nếu tồn tại đường đi từ s tới t , đường đi ngắn nhất tồn tại khi và chỉ khi không có đường đi từ s tới t chứa chu trình âm. Nếu có đường đi ngắn nhất, có một đường đi ngắn nhất đơn, vì có thể bỏ mọi chu trình không âm khỏi đường. Do đó khi có chu trình âm đạt được từ nguồn, bài toán nên trả ra chu trình âm thay vì một cây đường đi ngắn nhất.

Một cây đường đi ngắn nhất gốc s là cây phủ có gốc s , trong đó đường từ s tới mỗi đỉnh là ngắn nhất. Nếu T là cây có gốc s và $\text{distance}(v)$ là độ dài đường trong T từ s tới v , thì T là cây đường đi ngắn nhất khi và chỉ khi mọi cạnh $[v, w]$ thỏa

$$\text{distance}(v) + \text{length}(v, w) \geq \text{distance}(w).$$

Điều kiện này vừa là phép kiểm $O(m)$, vừa gợi ý thuật toán cải thiện lặp.

Thuật toán gán nhãn của Ford giữ khoảng cách tạm $\text{dist}(v)$ và cha tạm $p(v)$. Ban đầu $\text{dist}(s) = 0$, các đỉnh khác là vô cực, và mọi cha là null. Khi có cạnh $[v, w]$ vi phạm

$$\text{dist}(v) + \text{length}(v, w) < \text{dist}(w),$$

ta giảm $\text{dist}(w)$ và đặt $p(w) = v$. Khi không còn cạnh vi phạm, các nhãn là khoảng cách ngắn nhất và cha tạo cây đường đi ngắn nhất trên phần đạt được từ s . Nếu quá trình tạo chu trình theo cha, chu trình đó có tổng độ dài âm.

Gán nhãn và quét

Phiên bản hiệu quả hơn là gán nhãn-quét. Mỗi đỉnh ở một trong ba trạng thái: chưa gán nhãn, đã gán nhãn, đã quét. Ban đầu chỉ s đã gán nhãn. Mỗi bước chọn một đỉnh đã gán nhãn v , quét mọi cạnh ra $[v, w]$, và nếu cạnh cải thiện $\text{dist}(w)$ thì cập nhật w và chuyển w sang trạng thái đã gán nhãn. Khi không còn đỉnh đã gán nhãn, thuật toán dừng. Hiệu quả phụ thuộc hoàn toàn vào thứ tự quét.

Với đồ thị phi chu trình, quét theo thứ tự topo. Mỗi đỉnh cần quét đúng một lần, và tổng thời gian là $O(m)$, kể cả tìm thứ tự topo. Đổi dấu độ dài cạnh biến bài toán đường đi dài nhất trong DAG thành đường đi ngắn nhất, đây là lõi của phân tích PERT.

Nếu mọi cạnh không âm, quét theo thứ tự ngắn nhất trước: trong các đỉnh đã gán nhãn, chọn đỉnh có dist nhỏ nhất. Đây là thuật toán Dijkstra. Khi một đỉnh được quét, khoảng cách của nó đã tối ưu và sẽ không cần gán nhãn lại. Cài đặt giống Prim: heap chứa các đỉnh đã gán nhãn, khóa là dist . Với d -heap và $d = 2 + \lceil m/n \rceil$, thời gian là

$$O(m \log_{2+m/n} n).$$

Dùng tập không có thứ tự cho heap cho cận $O(n^2)$, phù hợp đồ thị dày. Nếu độ dài là số nguyên nhỏ, có thể dùng mảng bucket để đạt cận theo m , độ dài cạnh lớn nhất và khoảng cách lớn nhất.

Trong trường hợp tổng quát có cạnh âm nhưng không có chu trình âm đạt được, thứ tự quét theo chiều rộng cho thuật toán Bellman–Moore. Các đỉnh đã gán nhãn được giữ trong hàng đợi; mỗi lần lấy đỉnh đầu hàng đợi ra quét, và nếu một đỉnh được cải thiện nhưng chưa nằm trong hàng đợi thì thêm nó vào cuối. Chia quá trình thành các lượt: hết lượt k , mọi đường đi ngắn nhất dùng nhiều nhất k cạnh đã được phản ánh. Nếu không có chu trình âm đạt được, thuật toán dừng sau nhiều nhất $n - 1$ lượt, thời gian $O(nm)$.

Để phát hiện chu trình âm, có thể đếm lượt. Nếu sau lượt $n - 1$ hàng đợi vẫn không rỗng, tồn tại đỉnh v và số k sao cho $p^k(v) = v$; chu trình theo con trở cha là chu trình âm. Trong thực tế có các thứ tự lai giữa BFS và DFS có thể nhanh hơn trên một số đồ thị, nhưng cận xấu nhất có thể kém hơn.

Mọi cặp đỉnh

Nếu mọi độ dài không âm, chạy Dijkstra n lần cho n nguồn cho thời gian $O(nm \log_{2+m/n} n)$. Nếu có cạnh âm nhưng không có chu trình âm, dùng phép đổi trọng số của Edmonds–Karp/Johnson. Thêm nguồn mới s nối tới mọi đỉnh bằng cạnh độ dài 0, chạy thuật toán một nguồn tổng quát để lấy $\text{distance}(v)$, rồi đặt

$$\text{length}'(v, w) = \text{length}(v, w) + \text{distance}(v) - \text{distance}(w).$$

Mọi độ dài mới không âm, và mọi đường từ a tới b bị cộng cùng một lượng $\text{distance}(a) - \text{distance}(b)$, nên đường ngắn nhất được bảo toàn. Sau đó chạy Dijkstra cho từng nguồn rồi đổi ngược khoảng cách.

Với đồ thị dày, Floyd–Warshall đơn giản hơn. Duy trì $\text{dist}(v, w)$, là độ dài đường ngắn nhất từ v tới w mà các đỉnh trung gian chỉ thuộc tập đã xử lý. Ban đầu là độ dài cạnh trực tiếp, 0 trên đường chéo, vô cực nếu không có cạnh. Khi xử lý đỉnh u , cập nhật

$$\text{dist}(v, w) = \min\{\text{dist}(v, w), \text{dist}(v, u) + \text{dist}(u, w)\}.$$

Nếu $\text{dist}(u, u) < 0$, phát hiện chu trình âm. Để phục hồi đường đi, lưu thêm cha $p(v, w)$ của w trong cây đường đi ngắn nhất gốc v . Floyd chạy $O(n^3)$, dùng $O(n^2)$ bộ nhớ, thường tốt trên đồ thị dày nhưng không tận dụng được độ thưa.

Bất biến và ghi chú cài đặt của Chương 7

Tarjan dành khá nhiều chỗ cho các bất biến của phương pháp gán nhãn, vì đây là phần dễ bị xem nhẹ khi chỉ nhớ tên Bellman–Ford hoặc Dijkstra. Trong suốt thuật toán, nếu $\text{dist}(v)$ hữu hạn thì luôn tồn tại một đường thật từ s tới v có đúng độ dài đó. Khi thuật toán dừng, với mọi đường p từ s tới v , ta có

$$\text{length}(p) \geq \text{dist}(v),$$

nên nhãn hữu hạn chính là khoảng cách ngắn nhất. Điều này tách hai câu hỏi: *nhãn có nghĩa là một đường thật hay không* và *không còn đường nào tốt hơn hay không*.

Các con trở cha $p(v)$ cũng có bất biến riêng. Nếu $p(v) \neq \text{null}$ thì

$$\text{dist}(p(v)) + \text{length}(p(v), v) \leq \text{dist}(v),$$

và khi thuật toán dừng thì dấu bằng xảy ra. Tập cạnh cha hoặc tạo một cây gốc s trên các đỉnh có nhãn hữu hạn, hoặc chứa một chu trình. Nếu một bước giảm nhãn tạo chu trình cha thì tổng độ dài chu trình đó âm. Vì vậy phát hiện chu trình trong cây cha không chỉ là kiểm tra lỗi cài đặt; nó là cách trích ra chứng nhận chu trình âm.

Với quét theo hàng đợi trong trường hợp tổng quát, nguồn được quét ở lượt 0. Lượt $j > 0$ gồm các đỉnh đã nằm trong hàng đợi ở cuối lượt $j - 1$. Mỗi lượt tốn $O(m)$ vì mỗi đỉnh bị quét nhiều nhất một lần trong lượt. Nếu không có chu trình âm đạt được từ s , thì đến đầu lượt k , mọi đường ngắn nhất dùng nhiều nhất k cạnh đã được phản ánh vào nhãn; do đó thuật toán dừng muộn nhất sau lượt $n - 1$. Nếu sau lượt $n - 1$ hàng đợi vẫn còn đỉnh, lần theo cha từ một đỉnh bị quét ở lượt n sẽ gặp một đỉnh và cho chu trình âm.

Tarjan cũng nêu hai cách phát hiện chu trình âm sớm hơn đếm lượt. Khi xét cạnh $[v, w]$ có thể giảm nhãn w , ta có thể tìm w trong dãy tổ tiên của v , hoặc tìm v trong tập hậu duệ của w trong cây cha. Cách thứ nhất dễ viết bằng con trở cha nhưng có thể nâng cận xấu nhất lên $O(n^2m)$. Cách thứ hai cần lưu thêm thông tin cây, chẳng hạn một danh sách preorder đủ để kiểm tra quan hệ hậu duệ, nhưng nếu cài cẩn thận vẫn giữ được $O(nm)$.

Với Dijkstra, chương nhấn mạnh rằng cài đặt gần như trùng với Prim; khác biệt là khóa của heap là nhãn khoảng cách thay vì chi phí cạnh rẽ nhánh vào cây. Ngoài d -heap và tập không thứ tự, nguồn còn nhắc tới mảng bucket cho độ dài cạnh nguyên nhỏ, đặt cận theo $O(m + d)$ với d là khoảng cách lớn nhất từ nguồn và cần $O(m + L)$ bộ nhớ với L là độ dài cạnh lớn nhất. Chương cũng ghi lại hai biến thể: tìm đường ngắn nhất một cặp bằng tìm kiếm hai chiều trên đồ thị không âm, và bài toán đường cổ chai một nguồn, nơi ta tối thiểu hóa cạnh lớn nhất trên đường thay vì tổng độ dài.

Trong phần mọi cặp, phép đổi trọng số kiểu Johnson được viết như một phép thế năng. Nếu thêm nguồn mới và đặt $h(v) = \text{distance}(v)$, thì

$$\ell'(v, w) = \ell(v, w) + h(v) - h(w) \geq 0.$$

Với mọi đường $p : a \rightarrow b$, tổng trọng số mới bằng tổng trọng số cũ cộng $h(a) - h(b)$, một hằng số chỉ phụ thuộc hai đầu. Vì thế đường tối ưu được bảo toàn, còn Dijkstra trở nên hợp lệ trên từng nguồn.

Với Floyd, ngoài bảng khoảng cách có thể lưu $p(v, w)$, cha của w trong cây đường đi ngắn nhất gốc v . Khi cải thiện qua trung gian u , đặt

$$p(v, w) \leftarrow p(u, w),$$

vì đoạn cuối của đường mới từ v tới w chính là đoạn cuối của đường từ u tới w . Các bất biến của Tarjan gồm: đường được lưu có đúng độ dài $\text{dist}(v, w)$, chỉ dùng đỉnh trung gian đã xử lý, là đường đơn khi $v \neq w$, là chu trình đơn khi $v = w$, và thỏa

$$\text{path}(v, w) = \text{path}(v, p(v, w)) \& [w].$$

Nếu trong lúc xử lý có $\text{dist}(u, u) < 0$, các con trỏ cha tương ứng cho phép phục hồi một chu trình âm.

10 Chương 8: Luồng mạng

Luồng mạng được trình bày qua định nghĩa luồng, lát cắt và đường tăng. Từ định lý max-flow min-cut, các thuật toán có thể được xây bằng cách liên tục tìm đường tăng trong mạng dư.

Điểm nâng cấp là dùng *blocking flow*: thay vì tăng một đường mỗi lần, ta xây đồ thị mức và tìm một luồng chặn để loại bỏ cả một lớp đường tăng ngắn nhất. Sách cũng bàn cách tìm luồng chặn và chuyển sang luồng chi phí nhỏ nhất, nơi ngoài giá trị luồng còn cần tối ưu tổng chi phí.

Luồng, lát cắt và đường tăng

Mạng là đồ thị có hướng với nguồn s , đích t , và dung lượng dương $\text{cap}(v, w)$ trên mỗi cạnh. Một luồng f là hàm trên cặp đỉnh thỏa ba điều kiện: phản đối xứng $f(v, w) = -f(w, v)$, ràng buộc dung lượng $f(v, w) \leq \text{cap}(v, w)$, và bảo toàn luồng tại mọi đỉnh khác s, t . Giá trị $|f|$ là tổng luồng ròng rời nguồn. Bài toán max-flow cần tìm luồng giá trị lớn nhất.

Một lát cắt $X, \bar{X}$ có $s \in X, t \in \bar{X}$. Dung lượng lát cắt là tổng dung lượng các cạnh đi từ X sang $\bar{X}$. Luồng qua mọi lát cắt bằng đúng giá trị luồng, nên mọi luồng bị chặn trên bởi mọi lát cắt. Định lý max-flow min-cut nói rằng giá trị luồng cực đại bằng dung lượng lát cắt nhỏ nhất.

Với luồng f , dung lượng dư là $\text{res}(v, w) = \text{cap}(v, w) - f(v, w)$. Đồ thị dư chứa cạnh $[v, w]$ khi $\text{res}(v, w) > 0$. Một đường từ s tới t trong đồ thị dư là đường tăng; ta có thể tăng luồng dọc đường này một lượng bằng dung lượng dư nhỏ nhất trên đường. Nếu không còn đường tăng, tập đỉnh đạt được từ s trong đồ thị dư tạo một lát cắt có dung lượng bằng giá trị luồng, chứng minh luồng là cực đại.

Ford–Fulkerson lặp việc chọn một đường tăng bất kỳ. Với dung lượng nguyên, thuật toán dừng và sinh luồng nguyên. Nhưng chọn đường tùy ý có thể rất chậm, thậm chí không hội tụ tốt với dung lượng thực. Một lựa chọn tốt hơn là đường tăng có dung lượng lớn nhất; với dung lượng nguyên lớn nhất c , cần $O(m \log c)$ lần tăng, nhưng mỗi lần phải giải bài toán đường cổ chai.

Tăng bằng luồng chặn

Edmonds–Karp chọn đường tăng ngắn nhất theo số cạnh. Dinic gom các đường ngắn nhất lại. Với luồng hiện tại, gán mức của mỗi đỉnh là khoảng cách từ s trong đồ thị dư. Đồ thị mức giữ các cạnh $[v, w]$ thỏa $\text{level}(w) = \text{level}(v) + 1$. Nó chứa mọi đường tăng ngắn nhất và là DAG.

Một luồng là luồng chặn nếu mọi đường từ s tới t có ít nhất một cạnh bão hòa. Dinic lặp: xây đồ thị mức, tìm một luồng chặn trên đó, rồi cộng vào luồng hiện tại. Sau mỗi bước, khoảng cách mức của t tăng nghiêm hoặc t không còn đạt được, nên có nhiều nhất $n - 1$ bước chặn. Trên mạng đơn vị, số bước còn nhỏ hơn, $O(\sqrt{n})$, và cũng có cận theo $\sqrt{m}$ khi mọi dung lượng cạnh bằng một.

Nếu chỉ tăng từng đường trong đồ thị mức bằng DFS, mỗi lần bão hòa ít nhất một cạnh; cách cài Dinic cổ điển tìm luồng chặn trong $O(nm)$, cho max-flow $O(n^2m)$, và tốt hơn trên mạng đơn vị. Edmonds–Karp, nhìn như Ford–Fulkerson với đường tăng ngắn nhất từng cái, có nhiều nhất $(n - 1)m$ lần tăng và thời gian $O(nm^2)$.

Tìm luồng chặn

Karzanov đưa ra hướng bão hòa theo đỉnh thay vì theo cạnh. Tarjan trình bày phiên bản *wave*: dùng tiền-luồng, cho phép luồng dư dương tạm thời tích ở đỉnh. Đỉnh chưa chặn được cân bằng bằng cách đẩy luồng ra theo cạnh chưa bão hòa; nếu không đẩy hết được thì đánh dấu đỉnh là chặn. Đỉnh đã chặn được cân bằng bằng cách giảm luồng đi vào. Quét tiến theo thứ tự topo rồi quét lùi, mỗi vòng chặn thêm ít nhất một đỉnh, và tìm luồng chặn trong $O(n^2)$. Kết hợp với Dinic cho max-flow $O(n^3)$.

Malhotra–Kumar–Maheshwari dùng khái niệm thông lượng tiềm năng của đỉnh: tối thiểu giữa tổng dung lượng dư đi vào và đi ra. Mỗi bước chọn đỉnh có thông lượng tiềm năng nhỏ nhất, gửi lượng đó tiến tới t và lùi về s , rồi xóa cạnh bão hòa và đỉnh không còn nằm trên đường $s-t$. Ý tưởng đơn giản, nhưng khi cài đặt không nhất thiết đơn giản hơn wave và có thể chọn nhiều nút cổ chai hẹp.

Sleator–Tarjan tăng tốc cách DFS của Dinic bằng dynamic tree. Trong đồ thị mức, mỗi đỉnh giữ một cạnh hiện tại $[v, p(v)]$, các cạnh này tạo một rừng. Chi phí đỉnh là dung lượng dư của cạnh hiện tại nếu có cha, hoặc một hằng rất lớn nếu là gốc. Dynamic tree hỗ trợ tìm gốc, tìm chi phí nhỏ nhất trên đường, cộng chi phí trên đường, link và cut. Một lần tăng tương ứng với tìm chi phí nhỏ nhất trên đường từ s , trừ lượng đó khỏi cả đường, rồi cắt/xóa các cạnh bão hòa. Tổng cộng có $O(m)$ thao tác cây cho một luồng chặn, nên tìm luồng chặn trong $O(m \log n)$, và max-flow trong $O(nm \log n)$.

Luồng chi phí nhỏ nhất

Mỗi cạnh gờ có thêm chi phí đơn vị $\text{cost}(v, w)$, giả sử phản đối xứng. Chi phí của luồng là tổng chi phí đơn vị nhân lượng luồng. Một luồng là chi phí nhỏ nhất nếu trong các luồng cùng giá trị, nó có chi phí nhỏ nhất. Bài toán cần luồng cực đại có chi phí nhỏ nhất.

Định lý cơ bản: một luồng là chi phí nhỏ nhất khi và chỉ khi đồ thị dư của nó không có chu trình âm theo chi phí. Nếu có chu trình âm, đẩy luồng quanh chu trình làm giảm chi phí mà không đổi giá trị. Ngược lại, hiệu giữa một luồng chưa tối ưu và luồng tối ưu cùng giá trị phân rã thành các chu trình trong đồ thị dư, trong đó có chu trình âm.

Nếu luồng hiện tại đã là chi phí nhỏ nhất, tăng dọc một đường tăng có chi phí nhỏ nhất sẽ tạo ra luồng mới cũng chi phí nhỏ nhất trong các luồng cùng giá trị mới. Điều này dẫn tới thuật toán tăng theo đường chi phí nhỏ nhất, khi mạng ban đầu không có chu trình âm. Với dung lượng nguyên, thuật toán thực hiện nhiều nhất $|f^*|$ lần tăng.

Để tìm liên tiếp các đường tăng chi phí nhỏ nhất, dùng thuật toán đường đi ngắn nhất trên đồ thị dư. Sau mỗi lần tăng, đổi chi phí cạnh bằng thế năng khoảng cách:

$$\text{cost}'(v, w) = \text{cost}(v, w) + \text{dist}(v) - \text{dist}(w).$$

Chi phí mới không âm, cạnh trên đường vừa tăng có chi phí mới 0, nên các lần sau có thể dùng Dijkstra. Nếu chi phí ban đầu không âm hoặc đồ thị là DAG, lần đầu cũng nhanh; nếu không, dùng quét rộng tổng quát. Cận trên trong sách là

$$O(nm + m|f^*| \log_{2+m/n} n),$$

hoặc bỏ hạng nm khi chi phí ban đầu không âm hay đồ thị phi chu trình.

11 Chương 9: Ghép cặp

Chương cuối xét ghép cặp. Với đồ thị hai phía, ghép cặp cực đại có thể quy về luồng mạng hoặc xử lý bằng đường tăng luân phiên. Với đồ thị không hai phía, khó khăn chính là chu trình lẻ. Thuật toán blossom của Edmonds co các chu trình lẻ này thành siêu đỉnh để tiếp tục tìm đường tăng.

Sách nhấn mạnh cùng một mẫu: hiểu cấu trúc tổ hợp của bài toán, rồi chọn cấu trúc dữ liệu đủ mạnh để duy trì đúng thông tin cần thiết. Ghép cặp không hai phía là ví dụ rõ ràng cho việc một trở ngại lý thuyết, blossom, phải được biểu diễn cẩn thận trong thuật toán.

Ghép cặp hai phía và luồng mạng

Cho đồ thị vô hướng $G = [V, E]$ có trọng số trên cạnh. Một ghép cặp M là tập cạnh đôi một không chung đỉnh. Có bốn phiên bản: tối đa hóa kích thước hoặc trọng số, trên đồ thị hai phía hoặc tổng quát. Trường hợp hai phía có thể quy về luồng. Nếu phân hoạch hai phía là X, Y , thêm nguồn s , đích t , cạnh dung lượng 1 từ s tới mỗi $x \in X$, từ mỗi $y \in Y$ tới t , và từ x tới y ứng với cạnh $\{x, y\}$. Với bài toán trọng số, đặt chi phí cạnh $[x, y]$ là $-\text{weight}(x, y)$.

Một luồng nguyên trong mạng này tương ứng đúng với một ghép cặp: cạnh $\{x, y\}$ được chọn khi cạnh $[x, y]$ mang một đơn vị luồng. Do đó ghép cặp hai phía cực đại kích thước là max-flow trên mạng đơn vị, có thể giải bằng Dinic trong $O(\sqrt{n}m)$ theo cận của chương trước. Với trọng số, dùng luồng chi phí nhỏ nhất; dùng sau lần tăng cuối cùng có chi phí âm cho ta ghép cặp trọng số lớn nhất. Diễn giải trong ngôn ngữ ghép cặp chính là phương pháp Hungarian.

Đường luân phiên

Với một ghép cặp M , cạnh trong M gọi là cạnh ghép, cạnh ngoài M là cạnh tự do. Đỉnh incident với cạnh ghép là đỉnh đã ghép, ngược lại là đỉnh tự do. Một đường hoặc chu trình luân phiên có cạnh xen kẽ giữa ghép và tự do. Đường luân phiên là đường tăng nếu hai đầu của nó đều là đỉnh tự do. Đảo trạng thái các cạnh trên đường tăng làm kích thước ghép cặp tăng thêm một.

Nếu M^* là ghép cặp cực đại và $|M^*| - |M| = k$, hiệu đối xứng $M \triangle M^*$ phân rã thành các đường và chu trình luân phiên; trong đó có k đường tăng rời nhau, và ít nhất một đường có độ dài không quá $n/k - 1$. Vì vậy ghép cặp là cực đại khi và chỉ khi không còn đường tăng. Lập tìm đường tăng cho ta thuật toán cơ bản, tối đa $n/2$ lần tăng.

Trong trường hợp có trọng số, định nghĩa trọng số của đường luân phiên là tổng trọng số cạnh tự do trừ tổng trọng số cạnh ghép. Nếu M có trọng số lớn nhất trong các ghép cặp cùng kích thước và p là đường tăng có trọng số lớn nhất, thì tăng dọc p cho ghép cặp có trọng số lớn nhất trong các ghép cặp lớn hơn một cạnh. Đây là đối ứng của định lý tăng theo đường chi phí nhỏ nhất trong luồng chi phí nhỏ nhất.

Blossom

Trong đồ thị hai phía, tìm đường tăng bằng BFS luân phiên là đủ. Trong đồ thị không hai phía, chu trình lẻ phá vỡ cấu trúc phân lớp. Edmonds xử lý bằng blossom: một chu trình lẻ luân phiên có đúng một đỉnh gốc đặc biệt. Có thể co blossom thành một siêu đỉnh mà vẫn bảo toàn sự tồn tại của đường tăng; nếu đường tăng trong đồ thị co đi qua blossom, ta mở blossom ra theo một trong hai hướng luân phiên để nhận đường tăng thật.

Thuật toán tìm đường tăng duy trì một rừng luân phiên. Gốc là các đỉnh tự do và được đánh dấu chắn. Từ đỉnh chắn, xét cạnh chưa xét:

- nếu đi tới đỉnh lẻ, bỏ qua;
- nếu đi tới đỉnh chưa đạt, đánh dấu nó lẻ, rồi đánh dấu mate của nó chắn và thêm hai cạnh vào rừng;
- nếu đi tới đỉnh chắn thuộc cây khác, đã tìm được đường tăng;
- nếu đi tới đỉnh chắn trong cùng cây, chu trình lẻ tạo thành một blossom và được co lại.

Để cài co blossom hiệu quả, sách dùng union-find. Mỗi đỉnh co có một origin, là đỉnh đại diện trong rừng luân phiên. Khi gặp cạnh nối hai đỉnh chẵn trong cùng cây, tìm tổ tiên chung gần nhất trong rừng, rồi hợp các đỉnh trên hai nhánh vào blossom mới. Với các đỉnh lẻ bị co, lưu thêm bridge để sau này mở blossom và dựng đường tăng. Các thao tác makeset/link/find chỉ phôi thời gian; dùng union-find của Chương 2 cho thời gian $O(m\alpha(m, n))$ để phát hiện một đường tăng, hoặc tuyến tính $O(m)$ với thuật toán Gabow–Tarjan trong trường hợp đặc biệt.

Khi tìm được cạnh nối hai đỉnh chẵn ở hai cây khác nhau, đường tăng là đường từ đỉnh đó về gốc tự do của cây thứ nhất, cạnh nối, rồi đường về gốc tự do của cây thứ hai. Nếu đường đi qua blossom đã co, hàm dựng đường dùng origin và bridge để mở lại đúng nửa chu trình lẻ. Tổng thời gian dựng đường tăng tỉ lệ với độ dài đường, $O(n)$.

Lập thuật toán này cho ghép cặp không trọng số tổng quát cho cận $O(nm\alpha(m, n))$ hoặc $O(nm)$ tùy cài union-find. Những thuật toán sau đó của Even–Kariv và Micali–Vazirani kết hợp blossom với ý tưởng nhiều đường tăng ngắn nhất kiểu Hopcroft–Karp, đạt cùng bậc tốt nhất đã biết lúc đó cho hai phía và không hai phía không trọng số.

Với ghép cặp tổng quát có trọng số, Edmonds kết hợp tăng trọng số lớn nhất với co/mở blossom, nhưng blossom phải được duy trì qua nhiều lần tăng, chỉ mở hoặc co lại khi điều kiện đối ngẫu thay đổi. Gabow và Lawler đưa cài đặt xuống $O(n^3)$; Galil–Micali–Gabow dùng mở rộng của heap meldable để đạt $O(nm \log n)$. Chương kết thúc bằng việc đặt ghép cặp vào cùng dòng tư duy: đường tăng là nguyên lý tối ưu, còn blossom là cấu trúc dữ liệu/tổ hợp cần thiết để nguyên lý đó hoạt động ngoài đồ thị hai phía.

12 Ghi chú sử dụng

Cuốn sách này không phải sổ tay cài đặt từng dòng. Giá trị chính của nó là cách nhìn thống nhất: các thuật toán mạng nhanh không tách rời cấu trúc dữ liệu. Khi đọc trong bối cảnh thi thuật toán, nên đặc biệt chú ý các cặp sau:

- Kruskal đi cùng union-find;
- Prim và Dijkstra đi cùng hàng đợi ưu tiên;
- cây động đi cùng link-cut tree;
- luồng chặn đi cùng đồ thị mức và cấu trúc quét cạnh;
- ghép cặp tổng quát đi cùng blossom và đường luân phiên.

Ghi chú về bản dịch

PDF gốc có lớp văn bản tiếng Anh đọc được. Vì đây là một chuyên khảo dài, bản LaTeX hiện tại chuyển ngữ phần trước sách và tạo hướng dẫn chương bằng tiếng Việt, giữ các thuật ngữ thuật toán quan trọng để phục vụ tra cứu trong kho OI Public Library.